

STUDENT LABORATORY MANUAL

Module 4: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 4 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + FreeGLUT

Name	SERRANO, LIAM BRENT C.
Section	TN35
Github	https://github.com/TGOLiam/opengl-exercises

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: the 5-step vertex array conversion, `glDrawElements` indexed rendering, color arrays, interleaved arrays, and a final composite scene. Type or copy each one into your `SOURCE CODE\M4` folder, compile it (`Ctrl+Shift+B`), and run it (`F5`) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Each exercise gives you an Objective, a Task question describing exactly what your program must do, and Hints to point you in the right direction. Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

Use the shared `SOURCE CODE\VS Code` workspace described in the Module 1 Student Guide. Compile with `Ctrl+Shift+B` ("Build OpenGL Program"), then run with `F5` or from the terminal. All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with `g++` against `FreeGLUT`.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Immediate Mode Baseline (Before Conversion)

Concept: A plain glBegin()/glEnd() triangle — the starting point for Example 02's conversion.

Expected output: A solid green triangle, drawn the immediate-mode way.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 01 - Immediate Mode Baseline (Before Conversion)
 * -----
 * Concept: A plain glBegin()/glEnd() triangle, drawn in "immediate mode"
 * (one glVertex2f call issued at a time). This is the STARTING POINT that
 * Example 02 converts into a vertex array, so you can compare both
 * approaches side by side.
 */

void triangle() {
    glBegin(GL_TRIANGLES);
        glVertex2f(0.0f, 0.75f);
        glVertex2f(-0.75f, 0.0f);
        glVertex2f(0.75f, 0.0f);
    glEnd();
}

void display() { glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.16f, 0.72f, 0.08f); triangle();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex01 - Immediate Mode Triangle");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Example 02 — The Same Triangle, as a Vertex Array

Concept: The 5-step vertex array pattern: array -> glEnableClientState -> glVertexPointer -> glDrawArrays -> glDisableClientState.

Expected output: The identical green triangle from Example 01, now rendered from a vertex array.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 02 - Same Triangle, Converted to a Vertex Array
 * -----
 *
 * Concept: The 5-step vertex array pattern:
 * 1. Store the vertex coordinates in a plain array (GLfloat[]).
 * 2. glEnableClientState(GL_VERTEX_ARRAY) - tell OpenGL a vertex array
 *    will be used.
 * 3. glVertexPointer(size, type, stride, pointer) - tell OpenGL WHERE
 *    the data lives and how it's laid out.
 * 4. glDrawArrays(mode, first, count) - draw using the array data.
 * 5. glDisableClientState(GL_VERTEX_ARRAY) - turn the feature back off.
 */

void triangle() {
    // 1. initialize an array holding vertex coordinates GLfloat
    triangleVertex[] = {
        0.0f, 0.75f, 0.0f,          // z is always 0.0 for 2D primitives
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };

    // 2. activate the vertex array feature
    glEnableClientState(GL_VERTEX_ARRAY);

    // 3. tell OpenGL where the vertex data is glVertexPointer(3,
    GL_FLOAT, 0, triangleVertex);

    // 4. draw using the array
    glDrawArrays(GL_TRIANGLES, 0, 3);

    // 5. deactivate the vertex array feature
    glDisableClientState(GL_VERTEX_ARRAY);
}
```

```
void display() {
```

```

    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.16f, 0.72f, 0.08f); triangle();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex02 - Vertex Array Triangle"); glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 03 — GL_POINTS From a Vertex Array

Concept: Applying the 5-step pattern to GL_POINTS.

Expected output: Four large white points, one at each corner of a square.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 03 - GL_POINTS Drawn From a Vertex Array
 * -----
 * Concept: Applying the same 5-step vertex array pattern to GL_POINTS
 * instead of GL_TRIANGLES - only the array contents and glDrawArrays
 * mode change.
 */

void points() { glPointSize(20.0f);

    GLfloat pointVertex[] = {
        -0.75f, -0.75f, 0.0f,
        -0.75f, 0.75f, 0.0f,
        0.75f, 0.75f, 0.0f,
        0.75f, -0.75f, 0.0f
    };
}

```

```

    glVertexPointer(3, GL_FLOAT, 0, pointVertex);
    glDrawArrays(GL_POINTS, 0, 4);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f); points();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex03 - Vertex Array Points");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 04 — GL_LINES From a Vertex Array

Concept: A two-vertex array rendered with GL_LINES.

Expected output: A single thick horizontal yellow line.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 04 - GL_LINES Drawn From a Vertex Array
 * -----
 * Concept: A two-vertex array rendered with GL_LINES via the vertex
 * array pipeline.
 */

void line() {
    GLfloat lineVertex[] = {
        -0.8f, 0.0f, 0.0f,
        0.8f, 0.0f, 0.0f
    };
};

```

```

    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, lineVertex); glLineWidth(4.0f);
    glDrawArrays(GL_LINES, 0, 2);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 0.0f); line();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc, argv);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Ex04 - Vertex Array Line");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 05 — GL_QUADS From a Vertex Array

Concept: A 4-vertex array rendered with GL_QUADS.

Expected output: A solid blue filled square.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 05 - GL_QUADS Drawn From a Vertex Array
 * -----
 * Concept: A filled square, built from a 4-vertex array and rendered
 * with GL_QUADS.
 */

void square() {
    GLfloat quadVertex[] = {
        -0.5f, -0.5f, 0.0f,

```

```

        0.5f, 0.5f, 0.0f,
        -0.5f, 0.5f, 0.0f
    };

    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, quadVertex); glDrawArrays(GL_QUADS, 0, 4);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2f, 0.5f, 1.0f); square();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex05 - Vertex Array Square");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 06 — GL_POLYGON From a Vertex Array

Concept: A 5-vertex array rendered with GL_POLYGON.

Expected output: A solid orange filled pentagon.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 06 - GL_POLYGON Drawn From a Vertex Array
 * -----
 * Concept: A five-sided convex polygon (pentagon), built from a vertex
 * array and rendered with GL_POLYGON, using ALL specified vertices.
 */

```

```

    GLfloat pentagonVertex[] = { 0.0f,
        0.75f, 0.0f,
        -0.71f, 0.23f, 0.0f,
        -0.44f, -0.61f, 0.0f,
        0.44f, -0.61f, 0.0f,
        0.71f, 0.23f, 0.0f
    };

    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, pentagonVertex); glDrawArrays(GL_POLYGON,
    0, 5); glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 0.6f, 0.0f);
    pentagon();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex06 - Vertex Array Polygon");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 07 — Two Triangles, One Array, One Draw Call

Concept: Extending the array with more vertices lets one `glDrawArrays` render several separate primitives.

Expected output: Two separate green triangles, one above the other, drawn from a single 6-vertex array.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 07 - Two Triangles From ONE Array, ONE Draw Call
 * -----
 * Concept: Extending the vertex array with MORE vertices lets a single

```

```

* glDrawArrays() call render multiple separate primitives at once -
* here, two independent triangles (6 vertices, GL_TRIANGLES groups
* them 3 at a time).
*/

void triangles() {
    GLfloat triangleVertex[] = {
        // triangle 1 (upper) 0.0f,
        0.75f, 0.0f,
        -0.75f, 0.15f, 0.0f,
        0.75f, 0.15f, 0.0f,

        // triangle 2 (lower) - additional vertices for the 2nd triangle 0.0f, -0.15f, 0.0f,
        -0.75f, -0.75f, 0.0f,
        0.75f, -0.75f, 0.0f
    };

    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, triangleVertex);
    glDrawArrays(GL_TRIANGLES, 0, 6);    // 6 vertices to be rendered
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2f, 0.8f, 0.3f);
    triangles();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex07 - Multiple Primitives, One Array");
    glutDisplayFunc(display);
    glutMainLoop();
}

```

Example 08 — OpenGL Data Type Suffixes in Practice

Concept: glVertexPointer's `type` parameter with GLshort data instead of GLfloat.

Expected output: A pink triangle, its coordinates stored as GLshort and scaled down with glScalef.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>

```

```

#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 08 - OpenGL Data Type Suffixes in Practice -----
 *
 * Concept: glVertexPointer's "type" parameter is not limited to
 * GL_FLOAT. This example stores vertex coordinates as GLshort (16-bit
 * integer) instead of GLfloat, matching the "s" suffix from the OpenGL
 * data type table, to show the array-based pipeline works with several
 * underlying C types as long as `type` matches the array's declared type.
 */

void trianglesShort() {
    GLshort triangleVertex[] = { 0, 75,
                                -75, 0,
                                75, 0
    };

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(2, GL_SHORT, 0, triangleVertex);           // note: GL_SHORT, not GL_FLOAT
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() { glClear(GL_COLOR_BUFFER_BIT);
                glColor3f(0.9f, 0.2f, 0.6f);

                // GLshort coordinates above are scaled to fit within (-1,1) via glScalef glPushMatrix();
                glScalef(0.01f, 0.01f, 1.0f);
                trianglesShort(); glPopMatrix();

                glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex08 - GLshort Vertex Data Type");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 09 — glDrawElements(): Indexed Rendering, Basic Case

Concept: The same triangle as Example 02, this time drawn via glDrawElements with a simple {0,1,2} index array.

Expected output: The same green triangle, drawn through the indexed-rendering path.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 09 - glDrawElements(): Indexed Rendering
 * -----
 * Concept: Instead of glDrawArrays() walking the vertex array in
 * order, glDrawElements() reads a separate INDEX array telling it
 * which vertices to use, and in what order. Here the index array
 * simply selects vertices 0,1,2 - equivalent to Example 02, but using
 * the indexed drawing path.
 */

void triangle() {
    GLfloat triangleVertex[] = { 0.0f,
                                0.75f, 0.0f,
                                -0.75f, 0.0f, 0.0f,
                                0.75f, 0.0f, 0.0f
    };
    GLubyte indices[] = { 0, 1, 2 };

    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
GL_FLOAT, 0, triangleVertex);
    glDrawElements(GL_TRIANGLES, 3, GL_UNSIGNED_BYTE, indices);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() { glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.16f, 0.72f, 0.08f); triangle();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex09 - glDrawElements Basic");
    glutDisplayFunc(display);
```

```

    glutMainLoop();
    return 0;
}

```

Example 10 — glDrawElements(): Reusing Shared Vertices

Concept: A quad built from 4 unique vertices plus a 6-entry index array, instead of 6 (partly duplicated) vertices.

Expected output: A solid orange filled square, built from only 4 stored vertices.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 10 - glDrawElements(): Reusing Shared Vertices
 * -----
 * Concept: The real payoff of indexed rendering. A quad built from two
 * triangles normally needs 6 vertices with glDrawArrays (2 of them
 * duplicated). With glDrawElements, only the 4 UNIQUE corner
 * positions are stored once, and the index array tells OpenGL how to
 * reuse them to form both triangles - saving memory.
 */

void quadFromTriangles() {
    // Only 4 UNIQUE vertices needed, not 6
    GLfloat quadVertex[] = {
        -0.5f, -0.5f, 0.0f, // 0: bottom-left 0.5f, -0.5f, 0.0f,
        // 1: bottom-right 0.5f, 0.5f, 0.0f, // 2: top-right
        -0.5f, 0.5f, 0.0f // 3: top-left
    };

    // Triangle 1: 0,1,2      Triangle 2: 0,2,3 (vertices 0 and 2 are SHARED)
    GLubyte indices[] = { 0, 1, 2, 0, 2, 3 };

    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
        GL_FLOAT, 0, quadVertex);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, indices);
    glDisableClientState(GL_VERTEX_ARRAY);
}

```

```

    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.9f, 0.5f, 0.1f);
    quadFromTriangles(); glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex10 - glDrawElements Shared Vertices");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 11 — Vertex Array + Color Array Together

Concept: glColorPointer supplies per-vertex color alongside glVertexPointer's position data.

Expected output: A triangle with red, green, and blue corners, smoothly blending across its interior.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 11 - Combining a Vertex Array With a Color Array
 * -----
 *
 * Concept: glColorPointer works exactly like glVertexPointer, but
 * feeds per-vertex COLOR data instead of position data. Both arrays
 * are enabled and bound together, so glDrawArrays picks up matching
 * position[i] / color[i] pairs automatically.
 */

void triangle() {
    GLfloat vertices[] = { 0.0f,
        0.75f, 0.0f,
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };
    GLfloat colors[] = { 1.0f,    // red    - top vertex
        0.0f, 0.0f,    // green - bottom-left vertex
        0.0f, 1.0f, 0.0f, // blue  - bottom-right vertex
        0.0f, 0.0f, 1.0f
    };
}

```

```

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glVertexPointer(3, GL_FLOAT, 0, vertices); glColorPointer(3,
    GL_FLOAT, 0, colors);

    glDrawArrays(GL_TRIANGLES, 0, 3);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    triangle();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex11 - Vertex Array + Color Array");
    glutDisplayFunc(display);
    glutMainLoop();
}

```

Example 12 — Gradient Line Using a Color Array

Concept: Per-vertex colors from a color array interpolate across a primitive, same as Module 2's glColor3f approach.

Expected output: A thick line fading from yellow to purple.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 12 - Gradient Line Using a Color Array
 * -----
 * Concept: Per-vertex colors from a color array interpolate smoothly
 */

```

```

* across a primitive, exactly like per-vertex glColor3f calls inside
* glBegin/glEnd - only now the colors come from an array.
*/

void gradientLine() { GLfloat
    vertices[] = {
        -0.8f, 0.0f, 0.0f,
        0.8f, 0.0f, 0.0f
    };
    GLfloat colors[] = {
        1.0f, 1.0f, 0.0f,    // yellow
        0.6f, 0.0f, 0.8f    // purple
    };

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, vertices);
    glColorPointer(3, GL_FLOAT, 0, colors);
    glLineWidth(6.0f);
    glDrawArrays(GL_LINES, 0, 2);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    gradientLine();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex12 - Gradient Line via Color Array");
    glutDisplayFunc(display);
}

```

Example 13 — Four-Triangle Pinwheel, One Array, One Draw Call

Concept: Recreates the lecture's colored pinwheel using one vertex array + one color array + a single `glDrawArrays(GL_TRIANGLES, 0, 12)` call.

Expected output: A four-armed pinwheel, each arm fading red-green-blue.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>

```

```

#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 13 - Four-Triangle Pinwheel, One Array, One Draw Call -----
 *
 * Concept: Recreates the lecture's colored pinwheel (left/right/up/
 * down triangles, each fading red->green->blue) using a SINGLE vertex
 * array + SINGLE color array + ONE glDrawArrays(GL_TRIANGLES, 0, 12)
 * call, instead of four separate immediate-mode glBegin/glEnd blocks.
 */

GLfloat triangleVertices[] = {
    0.0f, 0.0f, 0.0f,      -0.5f, 0.10f, 0.0f,      -0.5f, -0.10f, 0.0f,      // left
    0.0f, 0.0f, 0.0f,      0.5f, 0.10f, 0.0f,      0.5f, -0.10f, 0.0f,      // right
    0.0f, 0.0f, 0.0f,      -0.10f, 0.50f, 0.0f,      0.10f, 0.50f, 0.0f, // up
    0.0f, 0.0f, 0.0f,      -0.10f, -0.50f, 0.0f, 0.10f, -0.50f, 0.0f      // down
};

GLfloat triangleColors[] = {
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f
};

void pinwheel() { glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, triangleVertices);
    glColorPointer(3, GL_FLOAT, 0, triangleColors);
    glDrawArrays(GL_TRIANGLES, 0, 12);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void display() { glClear(GL_COLOR_BUFFER_BIT);
    pinwheel();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex13 - Pinwheel: One Array, One Draw Call"); glutDisplayFunc(display);
    glutMainLoop();
}

```

```
return 0;
```

```
}
```

Example 14 — Full Scene: Pinwheel + Rectangle

Concept: Two separate vertex-array draw calls combined into one scene, mirroring the lecture's final sample program.

Expected output: The colored pinwheel with a solid green rectangle underneath it.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 14 - Full Scene: Pinwheel + Rectangle (Two Draw Calls)
 * -----
 *
 * Concept: A complete scene composed of TWO separate vertex-array draw
 * calls: the colored pinwheel (vertex array + color array), and a
 * solid green rectangle underneath (vertex array only, flat color).
 * Mirrors the lecture's final combined sample program.
 */

GLfloat triangleVertices[] = {
    0.0f, 0.0f, 0.0f,      -0.5f, 0.10f, 0.0f,      -0.5f, -0.10f, 0.0f,
    0.0f, 0.0f, 0.0f,      0.5f, 0.10f, 0.0f,      0.5f, -0.10f, 0.0f,
    0.0f, 0.0f, 0.0f,      -0.10f, 0.50f, 0.0f,      0.10f, 0.50f, 0.0f,
    0.0f, 0.0f, 0.0f,      -0.10f, -0.50f, 0.0f, 0.10f, -0.50f, 0.0f
};

GLfloat triangleColors[] = {
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f
};

GLfloat quadVertices[] = {
    -0.5f, -0.7f, 0.0f,
    0.5f, -0.7f, 0.0f,
    0.5f, -0.9f, 0.0f,
    -0.5f, -0.9f, 0.0f
};
```

```

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, triangleVertices); glColorPointer(3, GL_FLOAT,
    0, triangleColors); glDrawArrays(GL_TRIANGLES, 0, 12);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void rectangle() { glColor3f(0.0f, 1.0f,
    0.0f);
    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, quadVertices); glDrawArrays(GL_QUADS, 0, 4);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    pinwheel();
    rectangle(); glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(1024, 768);
    glutInitWindowPosition(200, 50);
    glutCreateWindow("My First OpenGL");
    glutDisplayFunc(display); glutMainLoop();
    return 0;
}

```

Example 15 — Indexed Pinwheel: Sharing the Center Vertex

Concept: The same pinwheel, drawn with `glDrawElements` so the shared center vertex is stored only once (12 -> 9 unique vertices).

Expected output: The same pinwheel shape as Example 13, this time built from an indexed vertex/color array pair.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>

```

```

using namespace std;

/*
 * Example 15 - Indexed Pinwheel: Sharing the Center Vertex
 * -----
 *
 * Concept: All four pinwheel triangles share the exact same center
 * point (0,0,0). Using glDrawElements, that shared vertex only needs
 * to be stored ONCE, and every triangle's index list simply points
 * back to it - reducing the unique vertex count from 12 to 9.
 */

GLfloat pinwheelVertices[] = {
    0.0f, 0.0f, 0.0f,          // 0: shared center
    -0.5f, 0.10f, 0.0f,        // 1: left-top
    -0.5f, -0.10f, 0.0f,       // 2: left-bottom
    0.10f, 0.0f, 0.5f,         // 3: right-top
    -0.10f, 0.0f, 0.5f,        // 4: right-bottom
    -0.10f, 0.50f, 0.0f,       // 5: up-left
    0.10f, 0.50f, 0.0f,        // 6: up-right
    -0.10f, -0.50f, 0.0f,      // 7: down-left
    0.10f, -0.50f, 0.0f,       // 8: down-right
};

GLfloat pinwheelColors[] = {
    1.0f, 1.0f, 1.0f,          // center - white
    1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, // left triangle outer colors
    0.0f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, // right triangle outer colors
    0.0f, 1.0f, 0.0f, 1.0f, 1.0f, 0.0f, // up triangle outer colors
    0.5f, 0.0f, 0.5f, 0.0f, 1.0f, 0.0f, // down triangle outer colors
};

GLubyte pinwheelIndices[] = {
    0, 1, 2, // left triangle (reuses shared center = index 0)
    0, 3, 4, // right triangle
    0, 5, 6, // up triangle
    0, 7, 8, // down triangle
};

void indexedPinwheel() {
    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, pinwheelVertices);
    glColorPointer(3, GL_FLOAT, 0, pinwheelColors);
    glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_BYTE, pinwheelIndices);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

```

```
void display() { glClear(GL_COLOR_BUFFER_BIT);  
    indexedPinwheel();  
    glFlush();
```

```

}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex15 - Indexed Pinwheel (Shared Center)");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 16 — Interleaved Arrays and the stride Parameter

Concept: Position and color interleaved into ONE array, read out using a non-zero stride.

Expected output: A red/green/blue triangle, identical to Example 11's result, sourced from a single interleaved array.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 16 - Interleaved Arrays and the "stride" Parameter
 * -----
 * Concept: So far, position and color have lived in SEPARATE arrays
 * (stride 0 = tightly packed, one array's own data only). Here they
 * are INTERLEAVED into a single array as [x,y,z,r,g,b, x,y,z,r,g,b,...]
 * and glVertexPointer/glColorPointer use a non-zero stride (the byte
 * offset to the NEXT vertex's data) plus a starting offset pointer to
 * read their own fields out of the same combined array.
 */

void interleavedTriangle() {
    // Each vertex: 3 position floats followed by 3 color floats GLfloat data[] = {
        // x,      y,      z,      r,      g,      b
        0.0f, 0.75f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.75f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
        0.75f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f
    };

    GLsizei stride = 6 * sizeof(GLfloat); // bytes from one vertex to the next

```

```

glEnableClientState(GL_VERTEX_ARRAY);
glEnableClientState(GL_COLOR_ARRAY);

glVertexPointer(3, GL_FLOAT, stride, data);           // position starts at offset 0
glColorPointer(3, GL_FLOAT, stride, data + 3);       // color starts 3 floats in

glDrawArrays(GL_TRIANGLES, 0, 3);

glDisableClientState(GL_VERTEX_ARRAY);
glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    interleavedTriangle(); glFlush();
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Ex16 - Interleaved Array with Stride");
    glutDisplayFunc(display);
    glutMainLoop(); return
    0
}

```

Example 17 — One Array, Multiple glDrawArrays 'first' Offsets

Concept: Reusing one big array across several glDrawArrays calls, each starting at a different 'first' index.

Expected output: Three separate triangles in a row, each a different color, all sourced from one shared array.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 17 - One Big Array, Multiple glDrawArrays "first" Offsets
 * -----
 *
 * Concept: A single vertex array can be reused across SEVERAL
 * glDrawArrays calls, each reading a different slice via the "first"
 * parameter - here, drawing three separate triangles stored back to

```

```

* back in one array, one glDrawArrays call per triangle, each with a
* different color and a different "first" starting index.
*/

GLfloat rowVertices[] = {
    -0.9f, -0.3f, 0.0f,      -0.7f, 0.3f, 0.0f,      -0.5f, -0.3f, 0.0f,      // triangle 0
    (indices 0-2)
    -0.15f, -0.3f, 0.0f,      0.05f, 0.3f, 0.0f,      0.25f, -0.3f, 0.0f,      // triangle 1
    (indices 3-5)
    0.5f, -0.3f, 0.0f,      0.7f, 0.3f, 0.0f,      0.9f, -0.3f, 0.0f      // triangle 2
    (indices 6-8)

};

void drawRow() {
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, rowVertices);

    glColor3f(1.0f, 0.2f, 0.2f);
    glDrawArrays(GL_TRIANGLES, 0, 3);      // first = 0

    glColor3f(0.2f, 1.0f, 0.2f);
    glDrawArrays(GL_TRIANGLES, 3, 3);      // first = 3

    glColor3f(0.2f, 0.4f, 1.0f);
    glDrawArrays(GL_TRIANGLES, 6, 3);      // first = 6

    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() { glClear(GL_COLOR_BUFFER_BIT);
    drawRow();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(700, 400);
    glutCreateWindow("Ex17 - One Array, Multiple Draw Calls");
    glutDisplayFunc(display);
    glutMainLoop(); return

```

Example 18 — glDrawElements() Across Multiple Shapes

Concept: One unique-vertex pool feeding two separate indexed draw calls for two different shapes.

Expected output: An orange quad and a blue triangle, built from two index lists into the same vertex pool.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 18 - glDrawElements() Across Multiple Shapes
 * -----
 *
 * Concept: A single unique-vertex pool feeding TWO separate indexed
 * draws - a triangle pair (quad) built with one index list, and a
 * standalone triangle built from three OTHER vertices in the same
 * pool, using a second glDrawElements call.
 */

GLfloat pool[] = {
    -0.9f, -0.5f, 0.0f, // 0
    -0.3f, -0.5f, 0.0f, // 1
    -0.3f, 0.5f, 0.0f, // 2
    -0.9f, 0.5f, 0.0f, // 3
    0.3f, -0.5f, 0.0f, // 4
    0.9f, -0.5f, 0.0f, // 5
    0.6f, 0.5f, 0.0f // 6
};

GLubyte quadIndices[] = { 0, 1, 2, 0, 2, 3 };
GLubyte triIndices[] = { 4, 5, 6 };

void drawShapes() {
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, pool);

    glColor3f(0.9f, 0.6f, 0.1f);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, quadIndices);

    glColor3f(0.1f, 0.6f, 0.9f);
    glDrawElements(GL_TRIANGLES, 3, GL_UNSIGNED_BYTE, triIndices);

    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() { glClear(GL_COLOR_BUFFER_BIT);
    drawShapes();
    glFlush();
}

```

```
int main(int argc, char** argv) {
```

```

    glutInit(&argc, argv);
    glutInitWindowSize(700, 400);
    glutCreateWindow("Ex18 - glDrawElements, Multiple Shapes");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 19 — Smoothly Shaded Quad Using a Color Array

Concept: Four vertices, four colors, one color array — OpenGL interpolates all four across the quad.

Expected output: A quad with red, green, blue, and yellow corners, smoothly blended.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 19 - Smoothly Shaded Quad Using a Color Array
 * -----
 * Concept: Four vertices, four different colors, one color array -
 * OpenGL interpolates all four across the quad's interior, exactly as
 * it would with four separate glColor3f calls inside glBegin/glEnd.
 */

void shadedQuad() { GLfloat
    vertices[] = {
        -0.6f, -0.6f, 0.0f,
        0.6f, -0.6f, 0.0f,
        0.6f, 0.6f, 0.0f,
        -0.6f, 0.6f, 0.0f
    };
    GLfloat colors[] = { 1.0f,
        0.0f, 0.0f,
        0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 1.0f,
        1.0f, 1.0f, 0.0f
    };

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

```

```

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    shadedQuad();
    glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(600, 500);
    glutCreateWindow("Ex19 - Smoothly Shaded Quad (Color Array)");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 20 — Capstone: An Efficient Composite Scene

Concept: A star (indexed triangle fan), a colored quad (vertex+color array), and a ground rectangle — every shape sourced from array data, no glVertex2f calls anywhere.

Expected output: A small scene: a yellow star, a multicolor quad, and a green ground strip.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__ #include
<GLUT/glut.h> #else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 20 - Capstone: An Efficient Composite Scene
 * -----
 * Concept: Combines everything from this module into one program:
 *   - A star-like GL_TRIANGLE_FAN built with glDrawElements, sharing
 *     one center vertex across all its slices (indexed rendering).
 *   - A colored quad rendered with a combined vertex array + color
 *     array in a single glDrawArrays call.
 *   - A plain ground rectangle rendered from its own small vertex array.
 * Every shape in the scene is drawn from array data - there is not a
 * single glVertex2f call left in this program.
 */

```

```

GLfloat starVertices[] = {
    0.0f, 0.35f, 0.0f,          // 0: shared center
    0.0f, 0.75f, 0.0f,          // 1
    0.15f, 0.45f, 0.0f,         // 2
    0.5f, 0.45f, 0.0f,          // 3
    0.2f, 0.25f, 0.0f,          // 4
    0.3f, -0.05f, 0.0f,         // 5
    0.0f, 0.15f, 0.0f,          // 6
    -0.3f, -0.05f, 0.0f,        // 7
    -0.2f, 0.25f, 0.0f,         // 8
    -0.5f, 0.45f, 0.0f,         // 9
    -0.15f, 0.45f, 0.0f,        // 10
    0.0f, 0.75f, 0.0f           // 11 (repeat of vertex 1 to close the fan)
};

GLubyte starIndices[] = { 0,1,2,3,4,5,6,7,8,9,10,11 };

// ---- Colored quad ----GLfloat
quadVertices[] = {
    -0.8f, -0.55f, 0.0f,
    -0.3f, -0.55f, 0.0f,
    -0.3f, -0.15f, 0.0f,
    -0.8f, -0.15f, 0.0f
};

GLfloat quadColors[] = { 1.0f,
    0.4f, 0.0f,
    0.0f, 0.8f, 1.0f,
    0.6f, 0.0f, 1.0f,
    1.0f, 1.0f, 0.0f
};

// ---- Ground rectangle ----GLfloat
groundVertices[] = {
    -0.9f, -0.8f, 0.0f,
    0.9f, -0.8f, 0.0f,
    0.9f, -0.95f, 0.0f,
    -0.9f, -0.95f, 0.0f
};

void star() {
    glColor3f(1.0f, 0.85f, 0.1f);
    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, starVertices);
    glDrawElements(GL_TRIANGLE_FAN, 12, GL_UNSIGNED_BYTE, starIndices);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void coloredQuad() { glEnableClientState(GL_VERTEX_ARRAY);

```

```
glEnableClientState(GL_COLOR_ARRAY); glVertexPointer(3,  
GL_FLOAT, 0, quadVertices); glColorPointer(3, GL_FLOAT, 0,  
quadColors);
```

```

    glDrawArrays(GL_QUADS, 0, 4);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void ground() {
    glColor3f(0.25f, 0.6f, 0.25f);
    glEnableClientState(GL_VERTEX_ARRAY); glVertexPointer(3,
    GL_FLOAT, 0, groundVertices); glDrawArrays(GL_QUADS, 0,
    4); glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    star();
    coloredQuad();
    ground(); glFlush();
}

int main(int argc, char** argv) { glutInit(&argc,
    argv); glutInitWindowSize(800, 600);
    glutCreateWindow("Ex20 - Capstone: Efficient Array-Based Scene");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard. Each exercise's "Task" line is the exact question you must answer with your program.

Easy Exercises

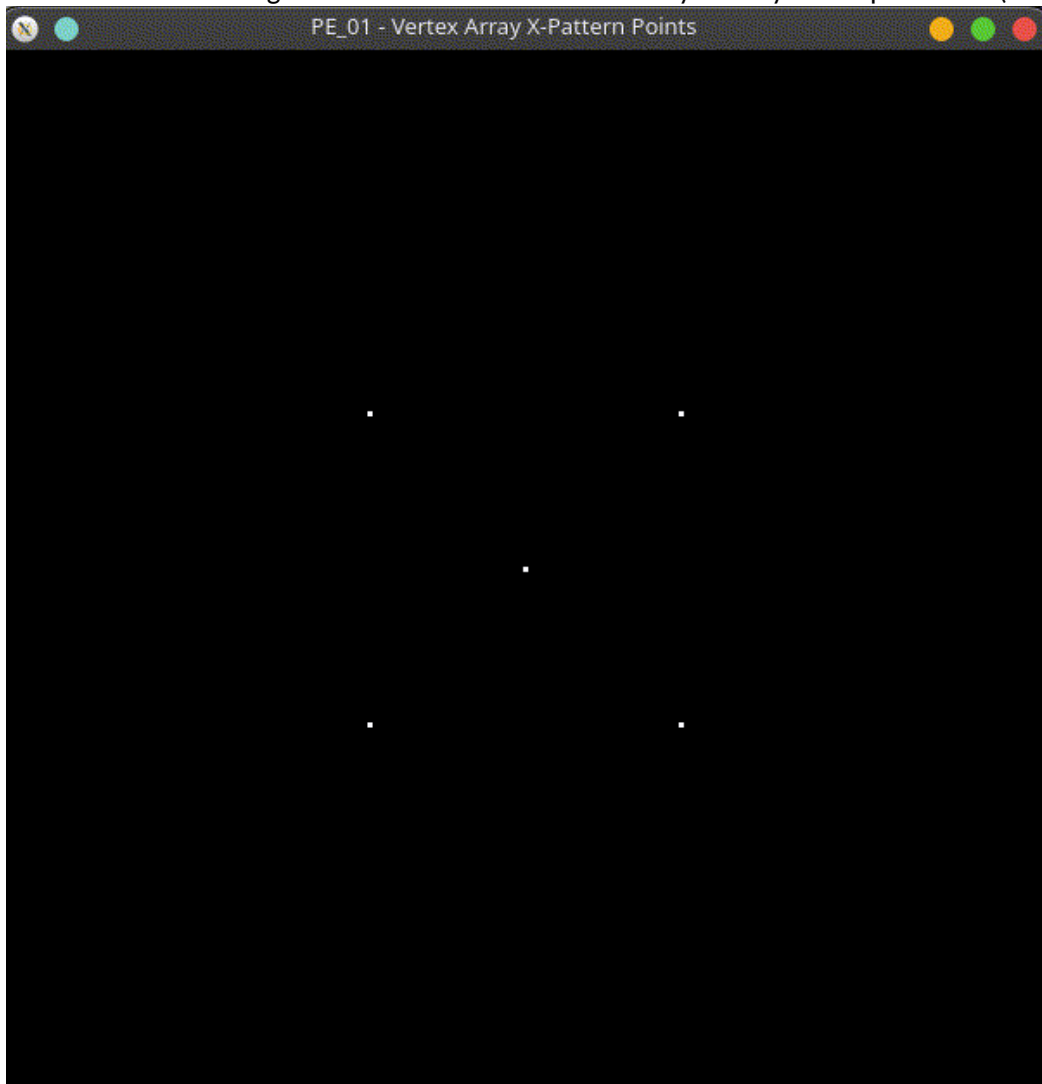
Exercise Q01 — Vertex Array X-Pattern Points [Easy]

Objective: Practice converting GL_POINTS to a vertex array using the 5-step pattern.

Task (Question): Convert an immediate-mode GL_POINTS block (5 points forming an X pattern with a center point) into a vertex array, using all 5 steps.

Hints:

- Set `glPointSize` BEFORE `glDrawArrays`, just as you would before `glBegin`.
- Double-check that `size` in `glVertexPointer` matches how many floats you use per vertex (2 or 3).



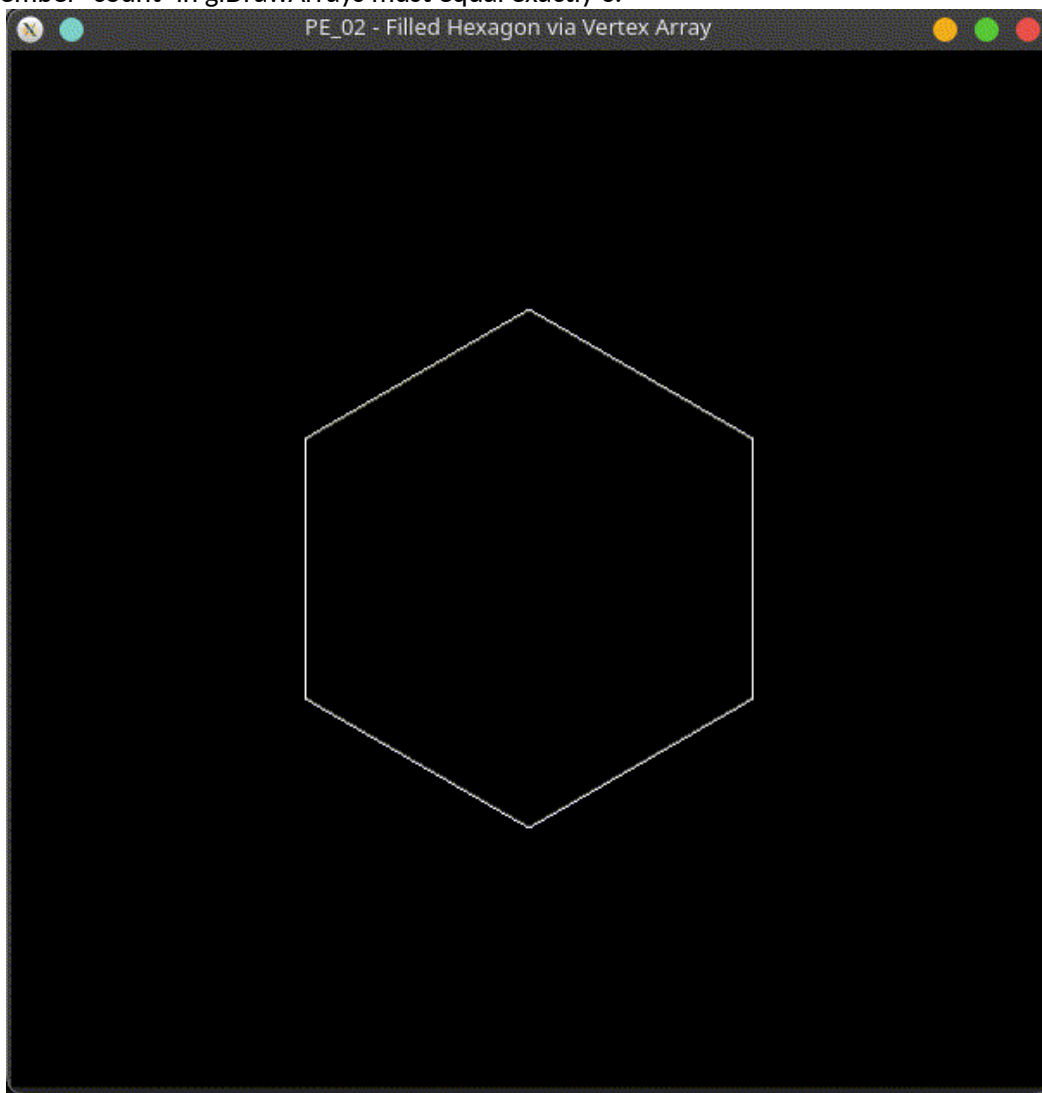
Exercise Q02 — Filled Hexagon via Vertex Array [Easy]

Objective: Practice GL_POLYGON with a larger vertex count than the lecture's pentagon example.

Task (Question): Draw a filled, six-sided hexagon using GL_POLYGON via a vertex array, with 6 unique vertices of your own choosing.

Hints:

- Plan your six (x, y) points on paper first, going consistently clockwise or counter-clockwise.
- Remember `count` in `glDrawArrays` must equal exactly 6.



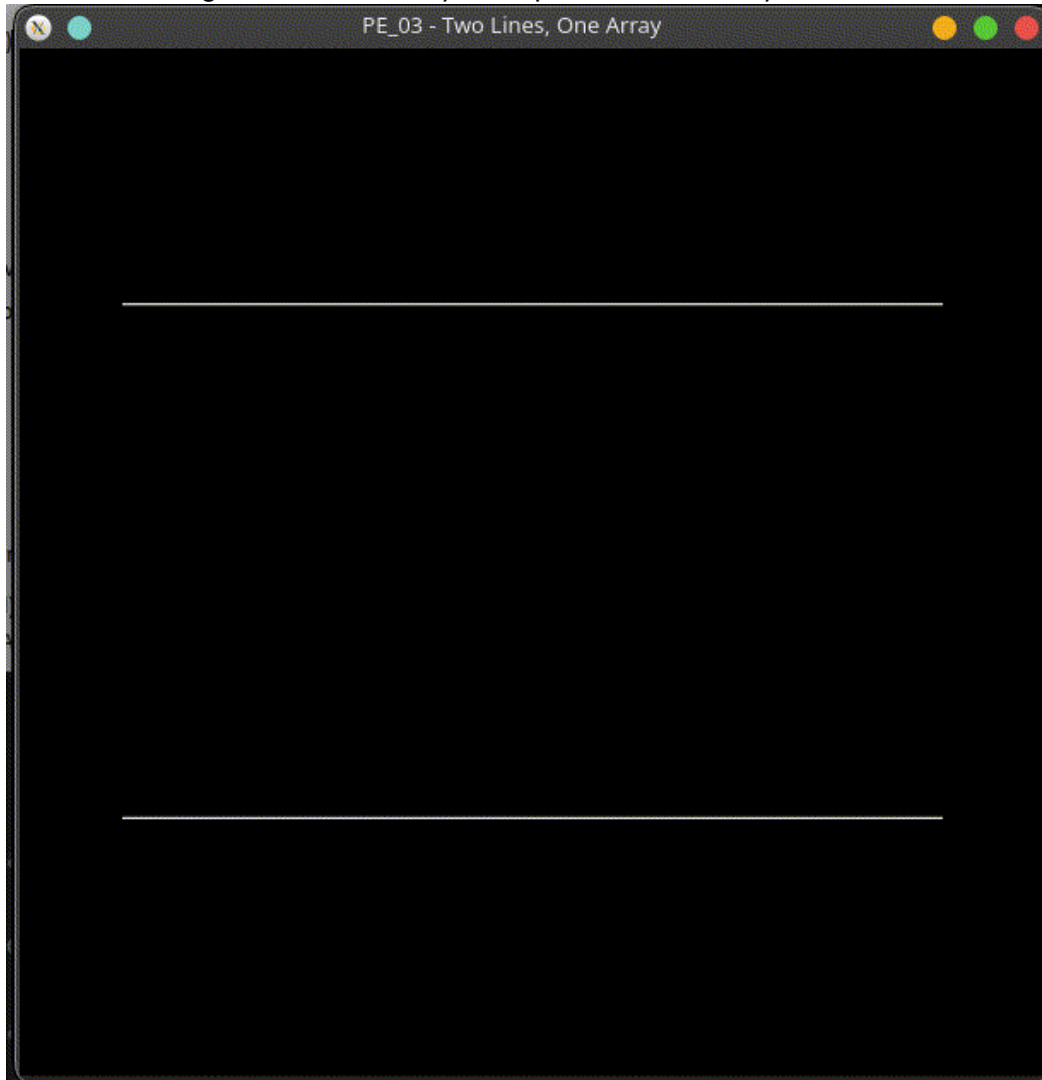
Exercise Q03 — Two Lines, One Array [Easy]

Objective: Practice packing multiple independent primitives into one vertex array.

Task (Question): Draw two separate GL_LINES segments from ONE vertex array and ONE glDrawArrays call (4 vertices total, grouped in pairs).

Hints:

- GL_LINES groups vertices in pairs — vertices 0-1 form segment one, vertices 2-3 form segment two.
- Make sure the two segments don't visually overlap so both are clearly visible.



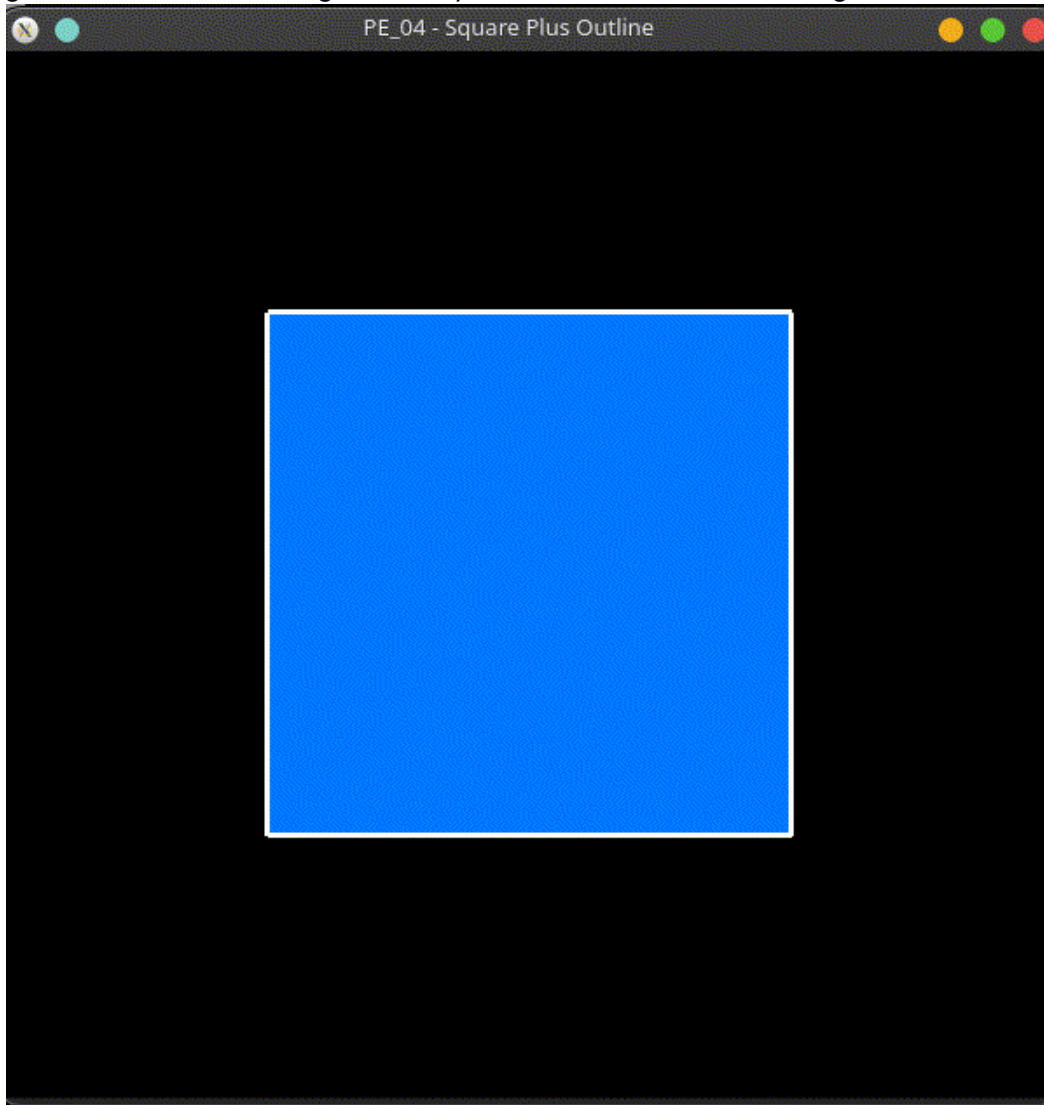
Exercise Q04 — Square Plus Outline, Same Array [Easy]

Objective: Practice reusing ONE vertex array across TWO different `glDrawArrays` calls.

Task (Question): Convert an immediate-mode filled square into a vertex array using `GL_QUADS`, then ALSO draw its outline using `GL_LINE_LOOP` with the SAME array (two draw calls, one array).

Hints:

- You only need to call `glEnableClientState/glVertexPointer` ONCE if both draw calls happen back to back.
- Change color between the two `glDrawArrays` calls so the outline is visible against the fill.



Exercise Q05 — GLint Vertex Data Type [Easy]

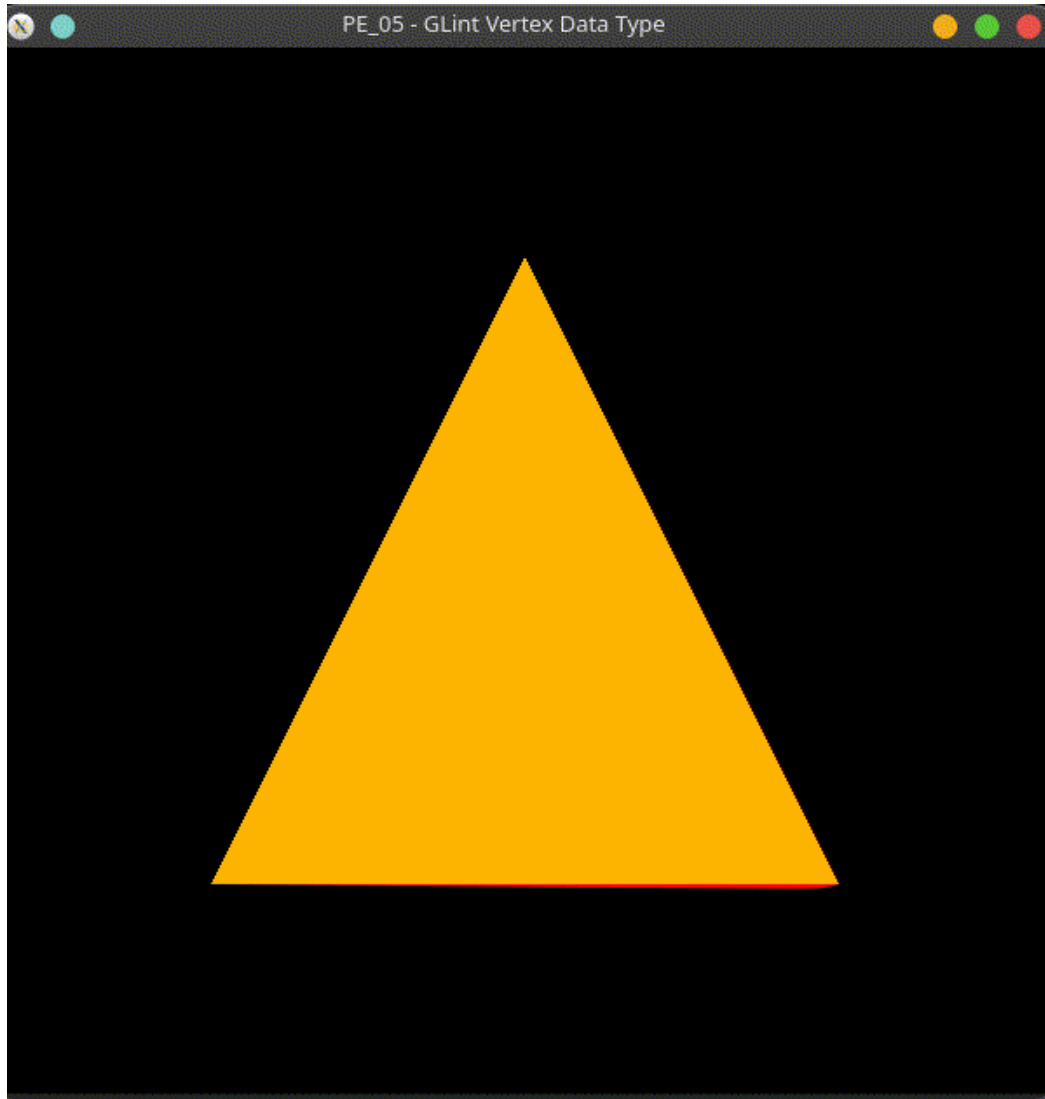
Objective: Practice using a non-GLfloat data type with `glVertexPointer`, per the OpenGL data type table.

Task (Question): Draw a triangle using a vertex array where the coordinates are stored as `GLint` (not

GLfloat), scaled down with `glScalef` so it still fits within the -1..1 viewport.

Hints:

- The `type` argument to `glVertexPointer` must be `GL_INT` to match a `GLint[]` array.
- Use `glPushMatrix/glScalef/glPopMatrix` around the draw call to bring large integer coordinates back into range.



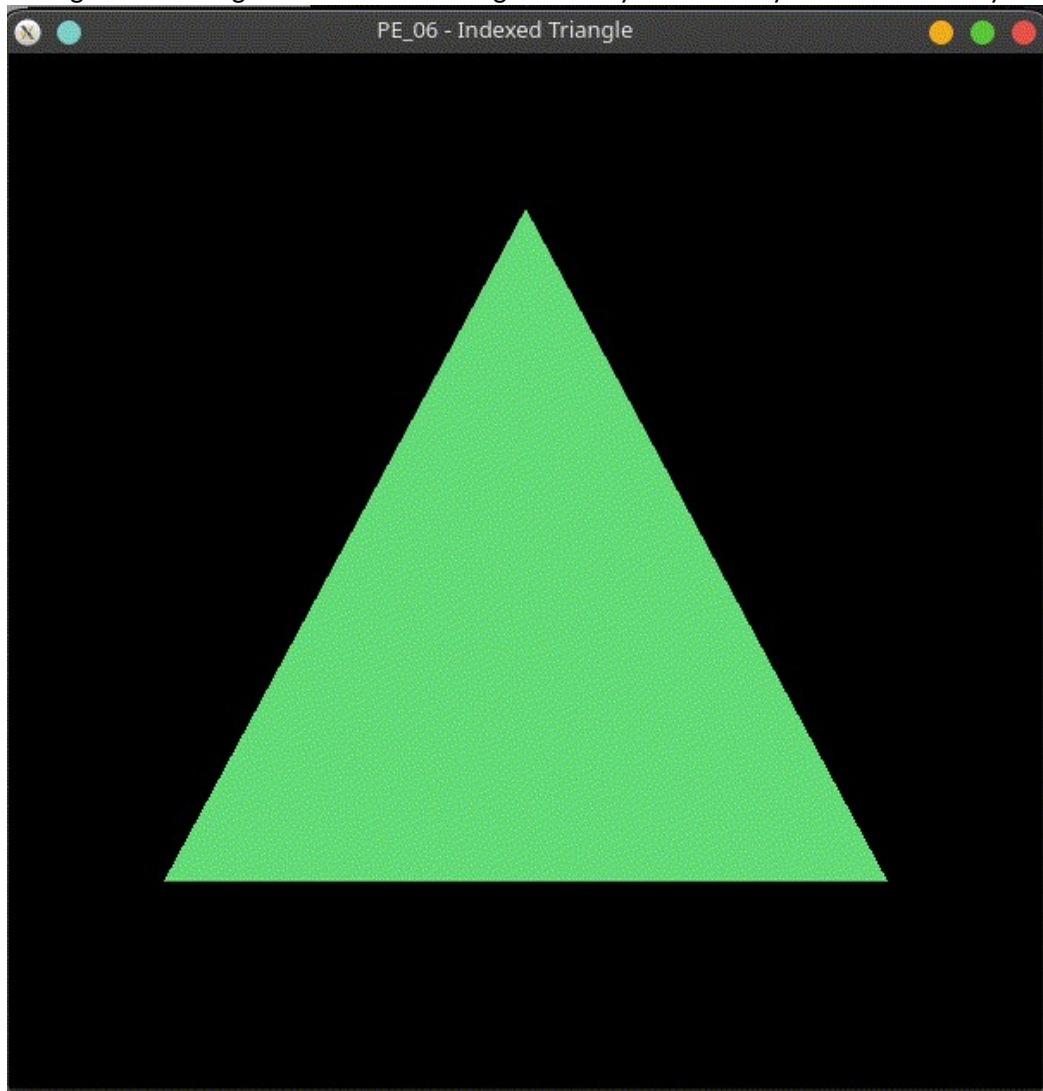
Exercise Q06 — `glDrawElements` Index Order [Easy]

Objective: Practice understanding that the INDEX ARRAY, not the array's storage order, controls draw order.

Task (Question): Draw a single triangle using `glDrawElements` with an index array, where the index order does NOT simply match the vertex array's storage order (e.g., indices {2, 0, 1} instead of {0, 1, 2}).

Hints:

- The rendered triangle should look identical either way — indices just control the ORDER vertices are visited, not which vertices exist.
- Try printing/commenting the intended winding order so you can verify it matches what you expect.



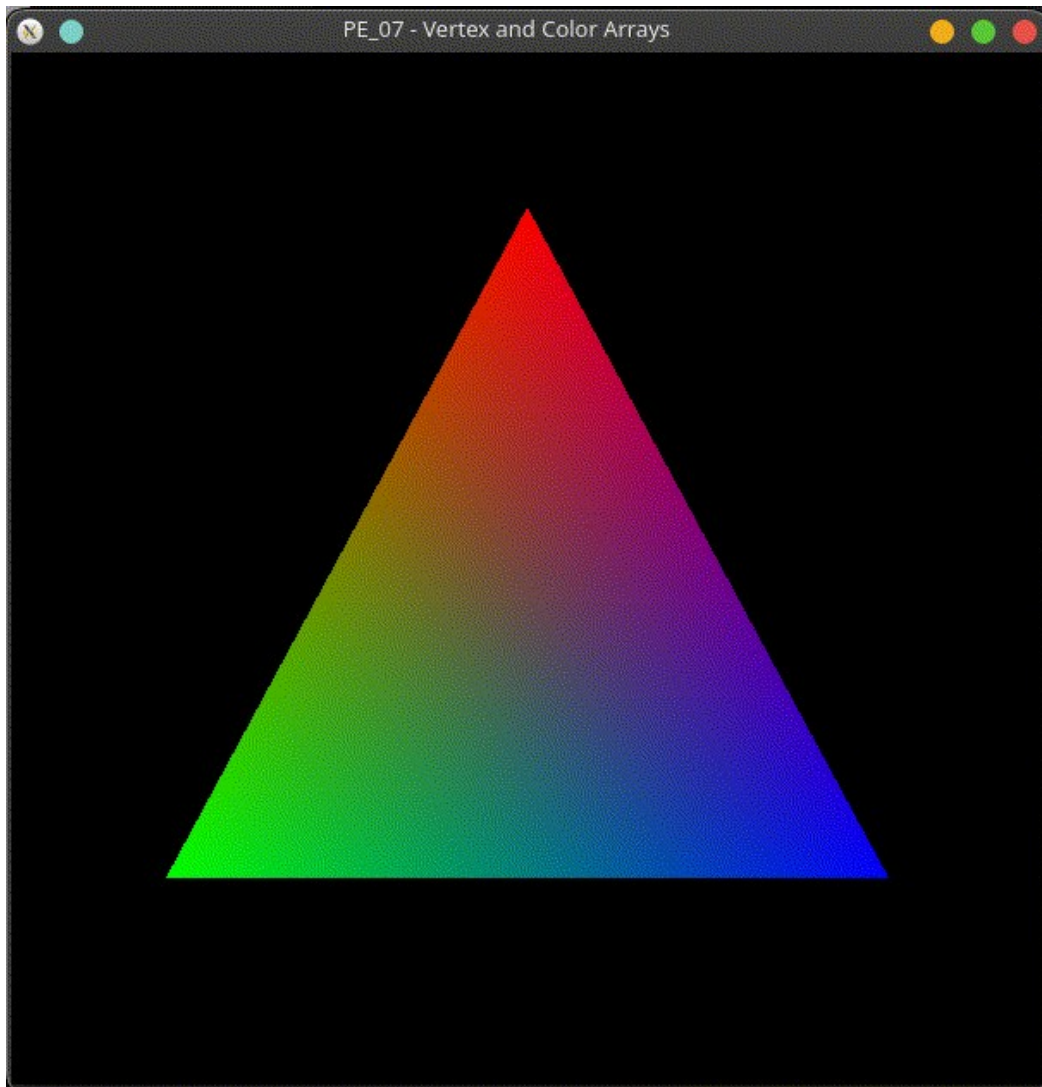
Exercise Q07 — Vertex + Color Array Triangle [Easy]

Objective: Practice the basic combination of `glVertexPointer` and `glColorPointer`.

Task (Question): Draw a triangle with a DIFFERENT color at each vertex, using a vertex array combined with a color array (not individual glColor3f calls).

Hints:

- Both GL_VERTEX_ARRAY and GL_COLOR_ARRAY must be enabled before the draw call.
- The i-th entry in the color array colors the i-th vertex in the vertex array — keep the two arrays in the same order.



Medium Exercises

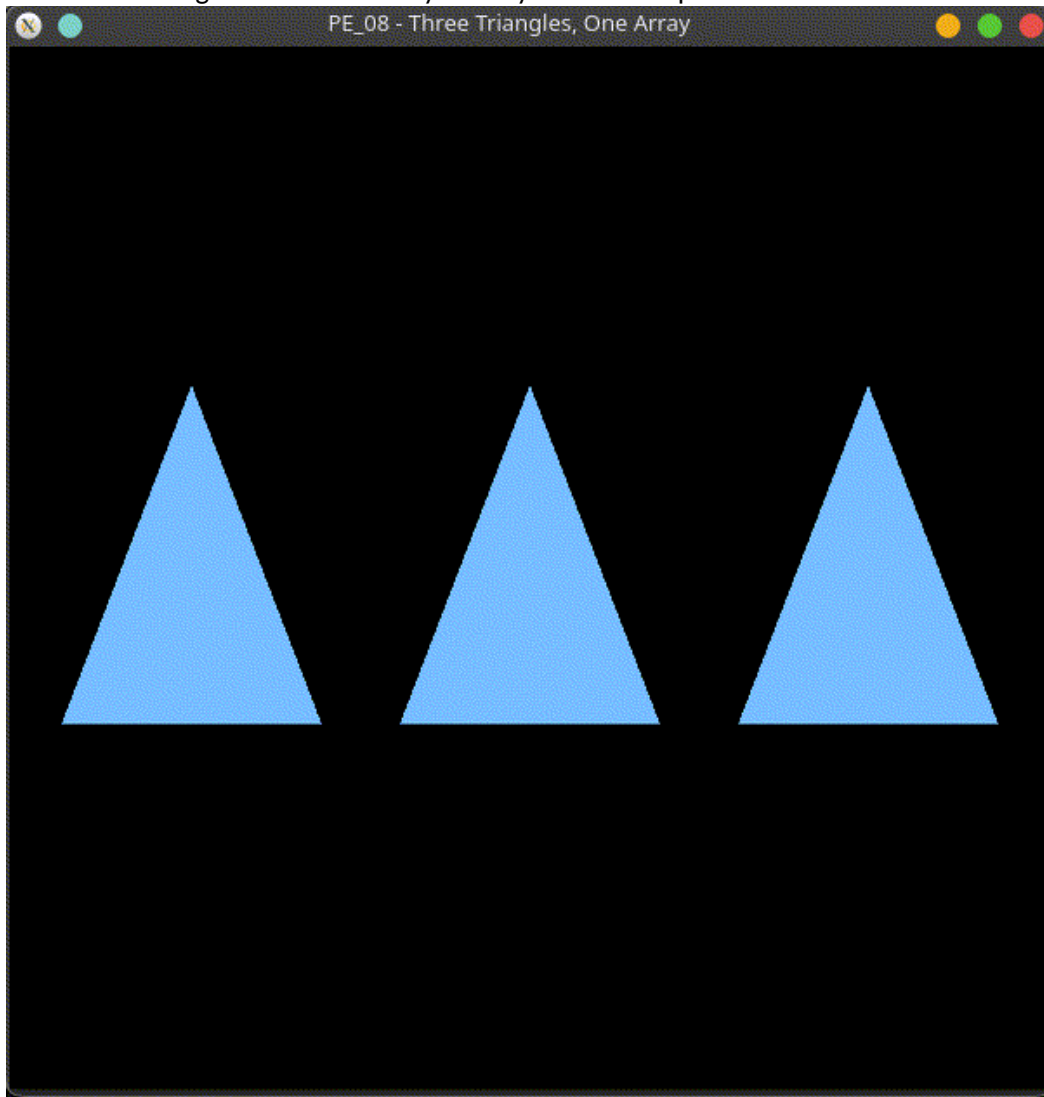
Exercise Q08 — Three Triangles, One Array [Medium]

Objective: Practice scaling up the 'multiple primitives, one array' pattern from Example 07.

Task (Question): Draw THREE separate triangles from one vertex array and ONE glDrawArrays call (9 vertices total).

Hints:

- GL_TRIANGLES groups vertices in sets of 3 — with 9 vertices you get exactly 3 triangles.
- Space the three triangles out horizontally so they don't overlap.



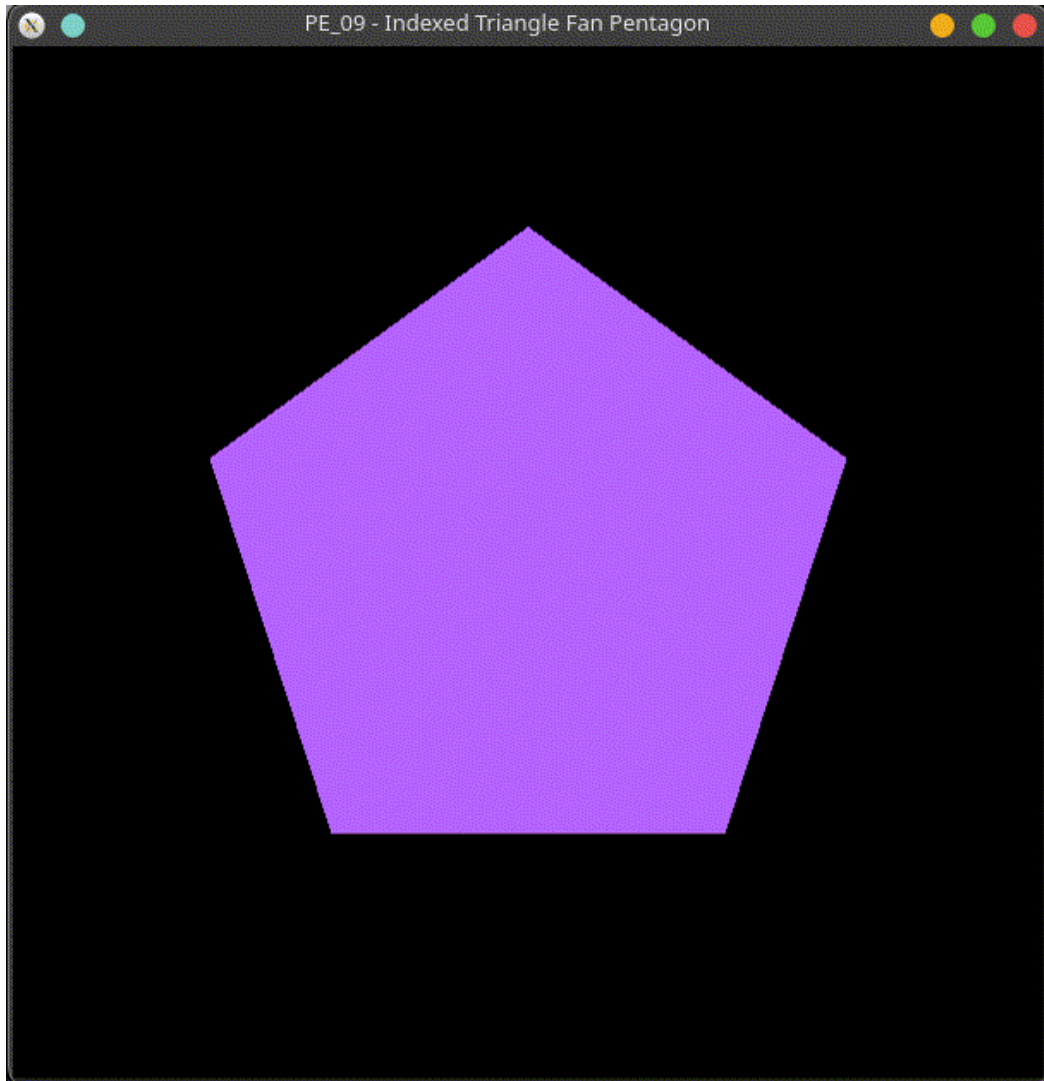
Exercise Q09 — Indexed Triangle Fan Pentagon [Medium]

Objective: Practice `glDrawElements` with `GL_TRIANGLE_FAN` and a shared center vertex.

Task (Question): Build a pentagon from `GL_TRIANGLE_FAN` using `glDrawElements`, where the shared center vertex is stored only ONCE in the vertex array.

Hints:

- A triangle fan's first vertex is the shared center — every triangle in the fan reuses it.
- Your index array should list the center index first, then the outer ring vertices in order, closing back to the first outer vertex.



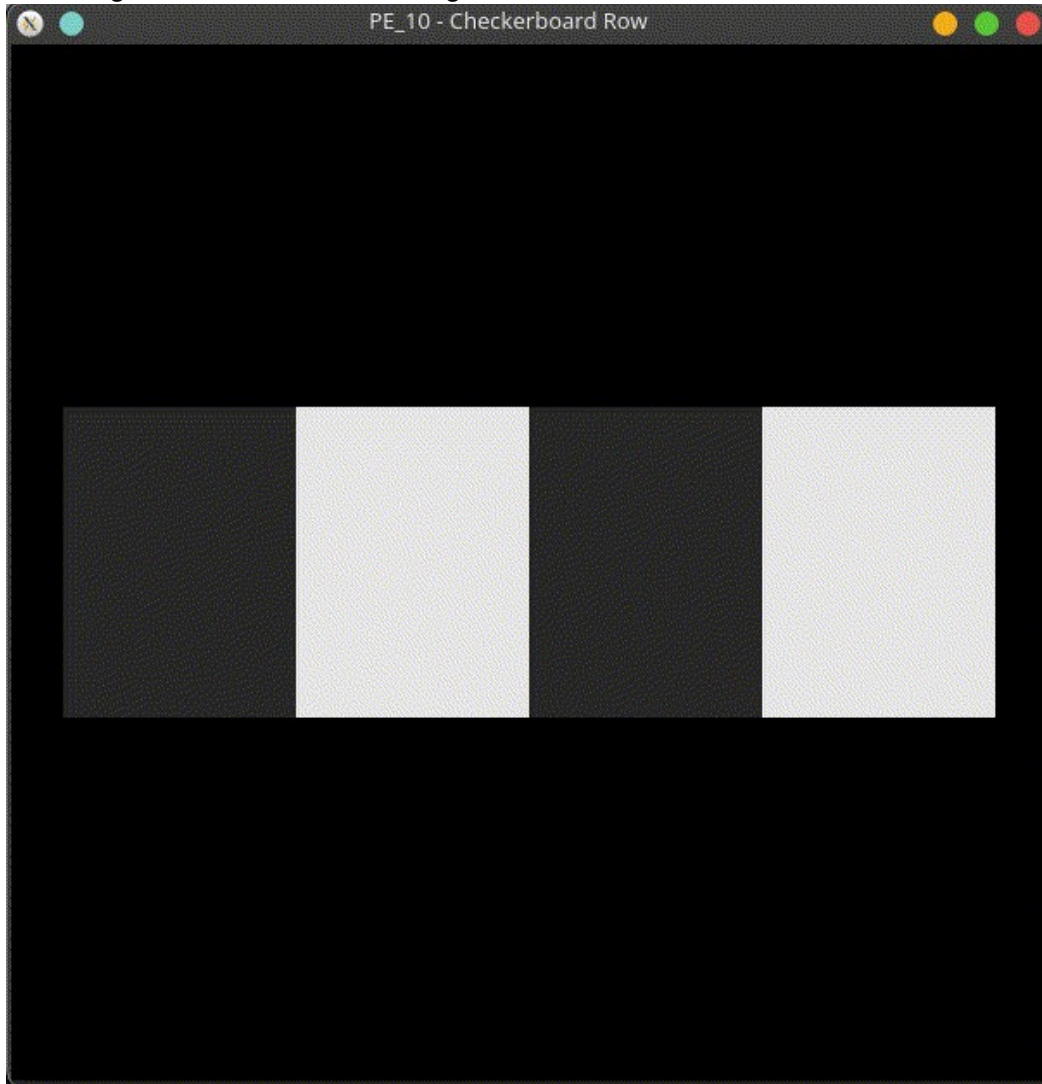
Exercise Q10 — Checkerboard Row via `glDrawElements` [Medium]

Objective: Practice building several `GL_QUADS` primitives from ONE shared vertex pool with SEPARATE index lists.

Task (Question): Build a checkerboard-style row of 4 quads using `glDrawElements`, alternating black and white, sharing vertices between neighboring quads where the edges touch.

Hints:

- Neighboring quads that share an edge can reuse the SAME two vertices from the pool — no need to store them twice.
- Alternate the glColor3f call between each glDrawElements call for the checkerboard effect.

**Exercise Q11 — Procedural Shaded Circle [Medium]**

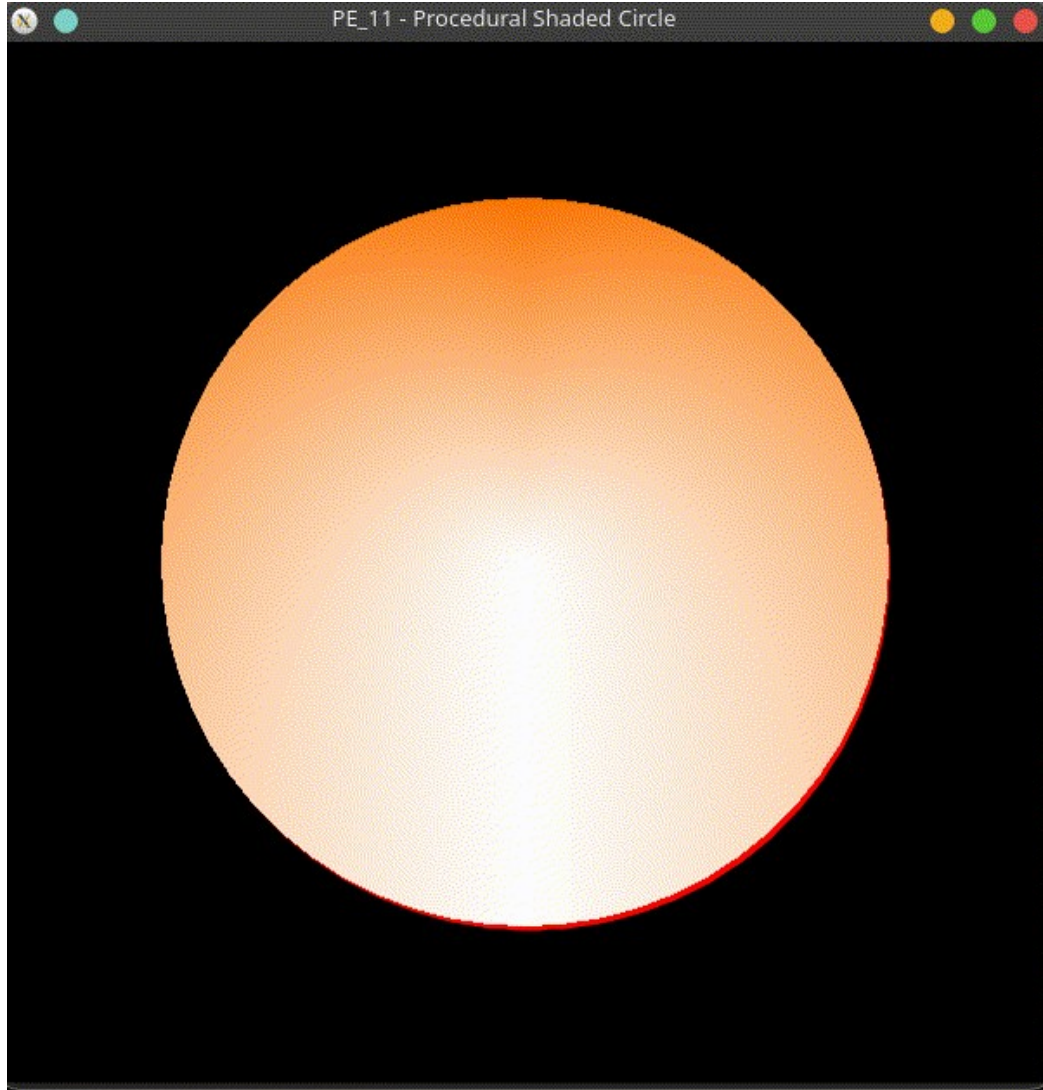
Objective: Practice generating vertex AND color array contents in a loop instead of hand-typing them.

Task (Question): Draw a smoothly shaded circle using GL_TRIANGLE_FAN, where BOTH the vertex array and the color array are filled in procedurally (using sin/cos and a loop), not hand-typed vertex by vertex.

Hints:

- Reserve arrays sized for (segments + 2) vertices: one center vertex, plus one extra to close the fan back to the start.

- Vary the color gradually as the loop index increases, for a visible color sweep around the circle.



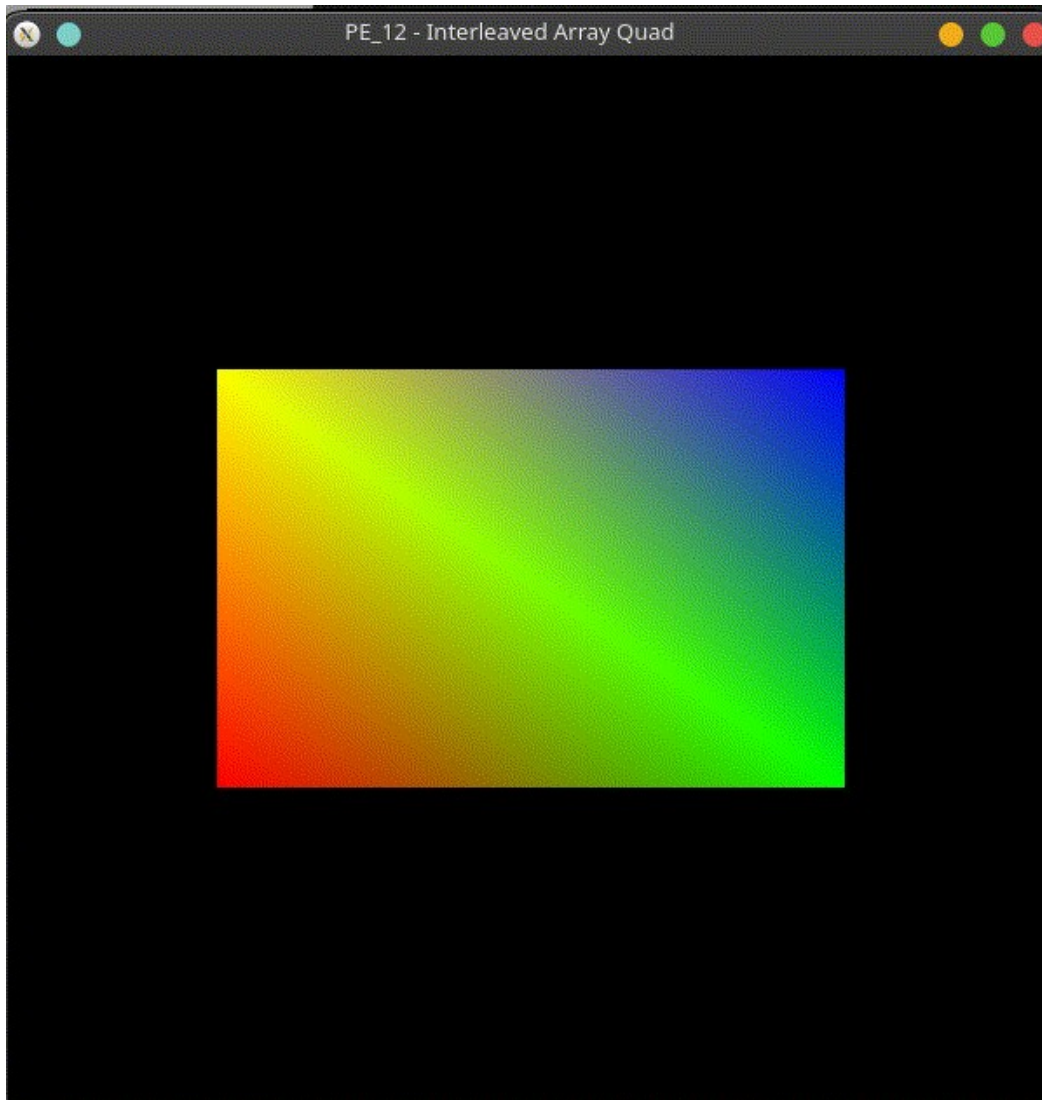
Exercise Q12 — Interleaved Array Quad [Medium]

Objective: Practice building and reading an interleaved position+color array with a non-zero stride.

Task (Question): Build an interleaved position+color array for a quad (like Example 16), and render it using stride-based `glVertexPointer`/`glColorPointer` calls.

Hints:

- Each vertex's 6 floats are x,y,z,r,g,b in that order — stride is $6 * \text{sizeof}(\text{GLfloat})$.
- The color pointer's starting address must be offset by 3 floats past the vertex pointer's start.



Exercise Q13 — Pinwheel via `glDrawElements` [Medium]

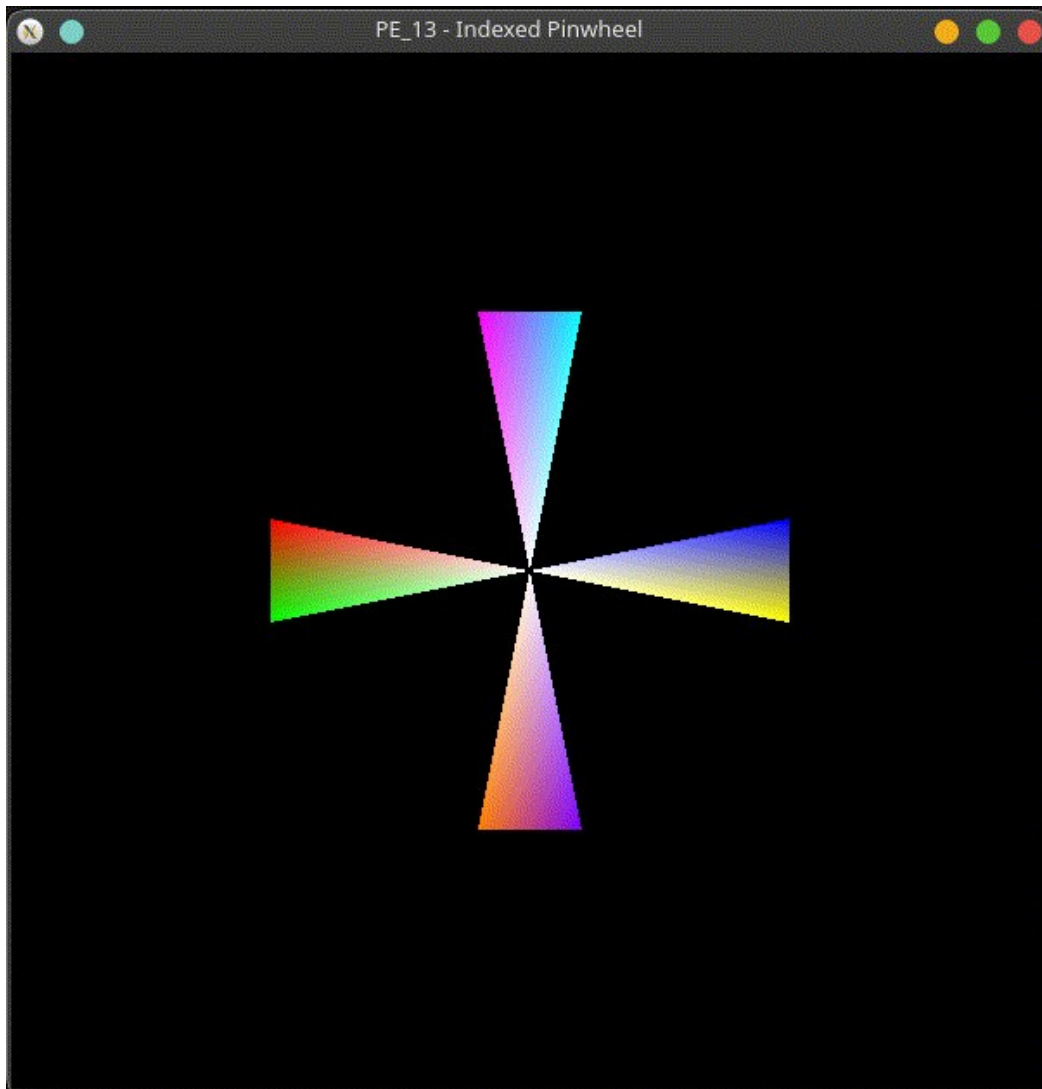
Objective: Practice re-implementing a known shape (the lecture's pinwheel) using indexed rendering instead of `glDrawArrays`.

Task (Question): Recreate the lecture's four-triangle pinwheel using `glDrawElements` **INSTEAD** of `glDrawArrays`, sharing the center vertex across all four triangles.

Hints:

- The center vertex (0,0,0) should appear only **ONCE** in the vertex array, no matter how many triangles use it.

- Your index array will have 12 entries (4 triangles x 3 indices), but your vertex array will have only 9 unique vertices.



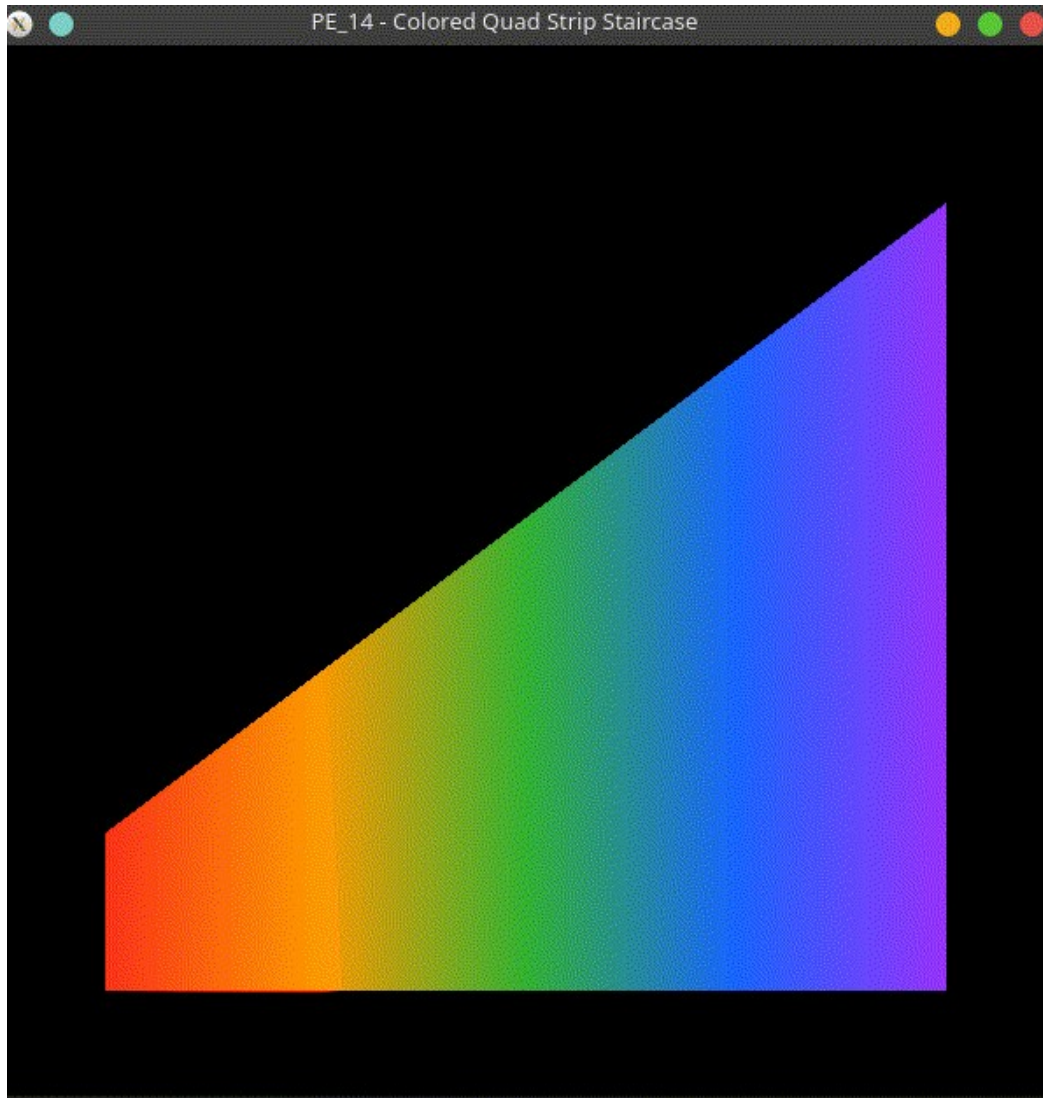
Exercise Q14 — Colored Quad Strip Staircase [Medium]

Objective: Practice combining an earlier primitive type (`GL_QUAD_STRIP` from Module 2) with vertex + color arrays.

Task (Question): Combine a `GL_QUAD_STRIP` 'staircase' shape with a vertex array AND a color array so each step is a different color, all drawn in ONE `glDrawArrays` call.

Hints:

- Remember `GL_QUAD_STRIP` vertices alternate bottom, top, bottom, top as you move across the strip.
- Give each step's bottom and top vertex the SAME color so each step reads as one solid color rather than blending into its neighbor.



Hard Exercises

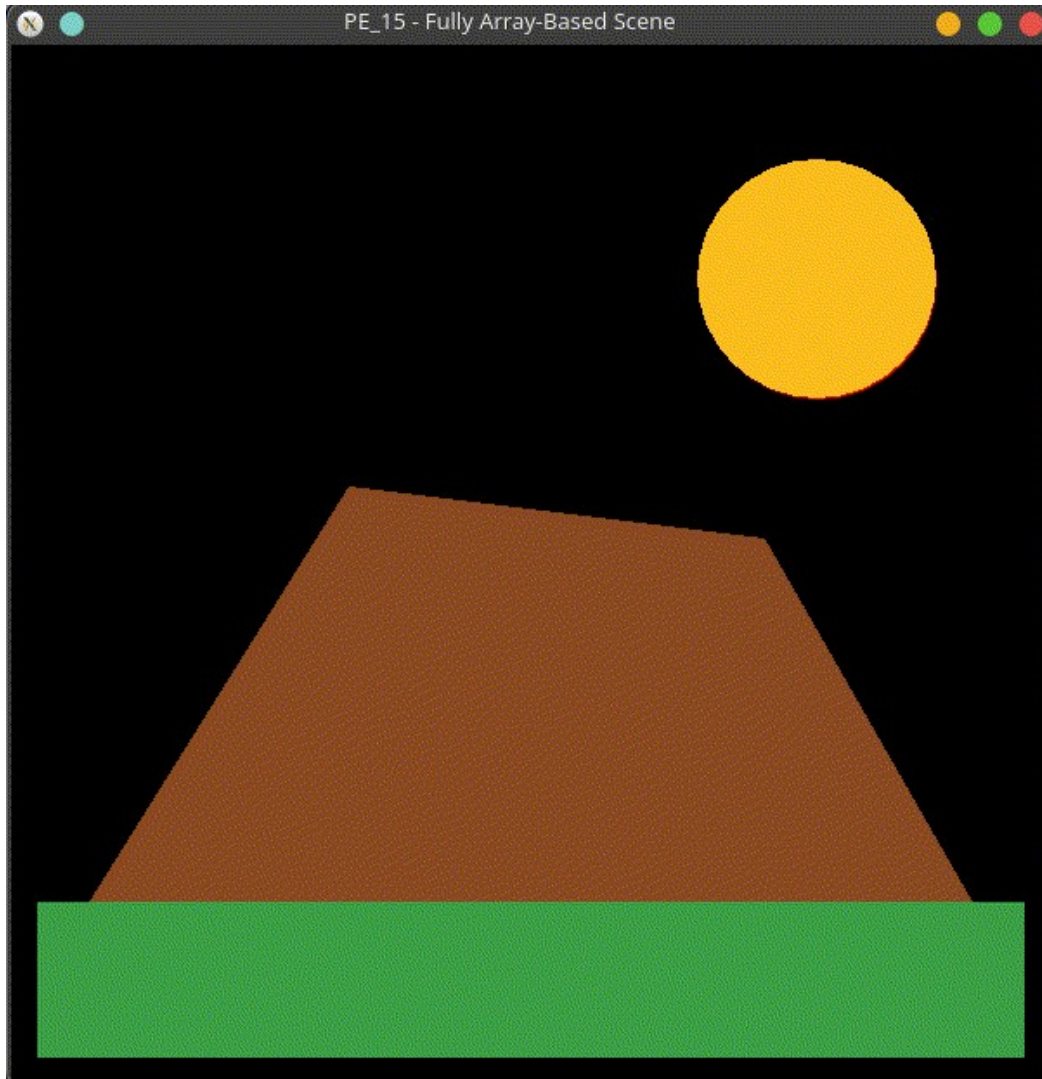
Exercise Q15 — Fully Array-Based Scene [Hard]

Objective: Practice building a multi-shape scene where every single shape uses vertex array data.

Task (Question): Build a full scene (at least 3 shapes: e.g., a sun, a mountain, and ground) where EVERY shape is drawn from vertex array data — no `glVertex2f` calls anywhere — mixing `glDrawArrays` and `glDrawElements` as appropriate for each shape.

Hints:

- A sun or star shape is a natural fit for `glDrawElements` with a shared center vertex.
- Simple shapes like the ground rectangle can stay as plain `glDrawArrays` calls — you don't need indexing everywhere.



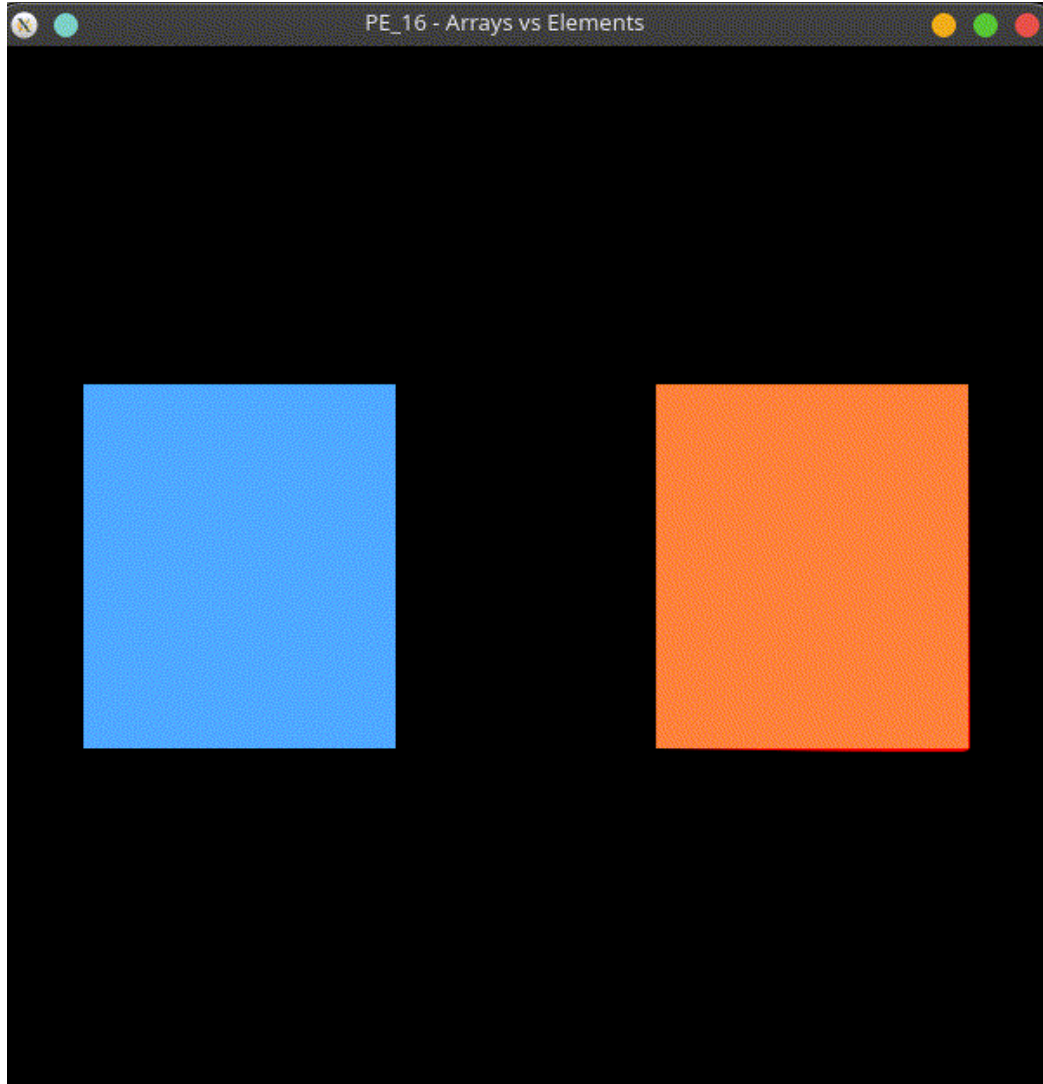
Exercise Q16 — `glDrawArrays` vs. `glDrawElements`, Side by Side [Hard]

Objective: Practice directly comparing the vertex counts of both approaches on the identical shape.

Task (Question): Draw the SAME quad twice, side by side: once with `glDrawArrays` (6 vertices, 2 duplicated), and once with `glDrawElements` (4 unique vertices + a 6-entry index array), so both approaches can be visually and numerically compared.

Hints:

- Use a comment above each array noting exactly how many floats/vertices are stored, so the savings are obvious at a glance.
- Give the two quads different colors so it's clear they are two separate draw calls, not one shape.



Exercise Q17 — Procedural Gear Shape [Hard]

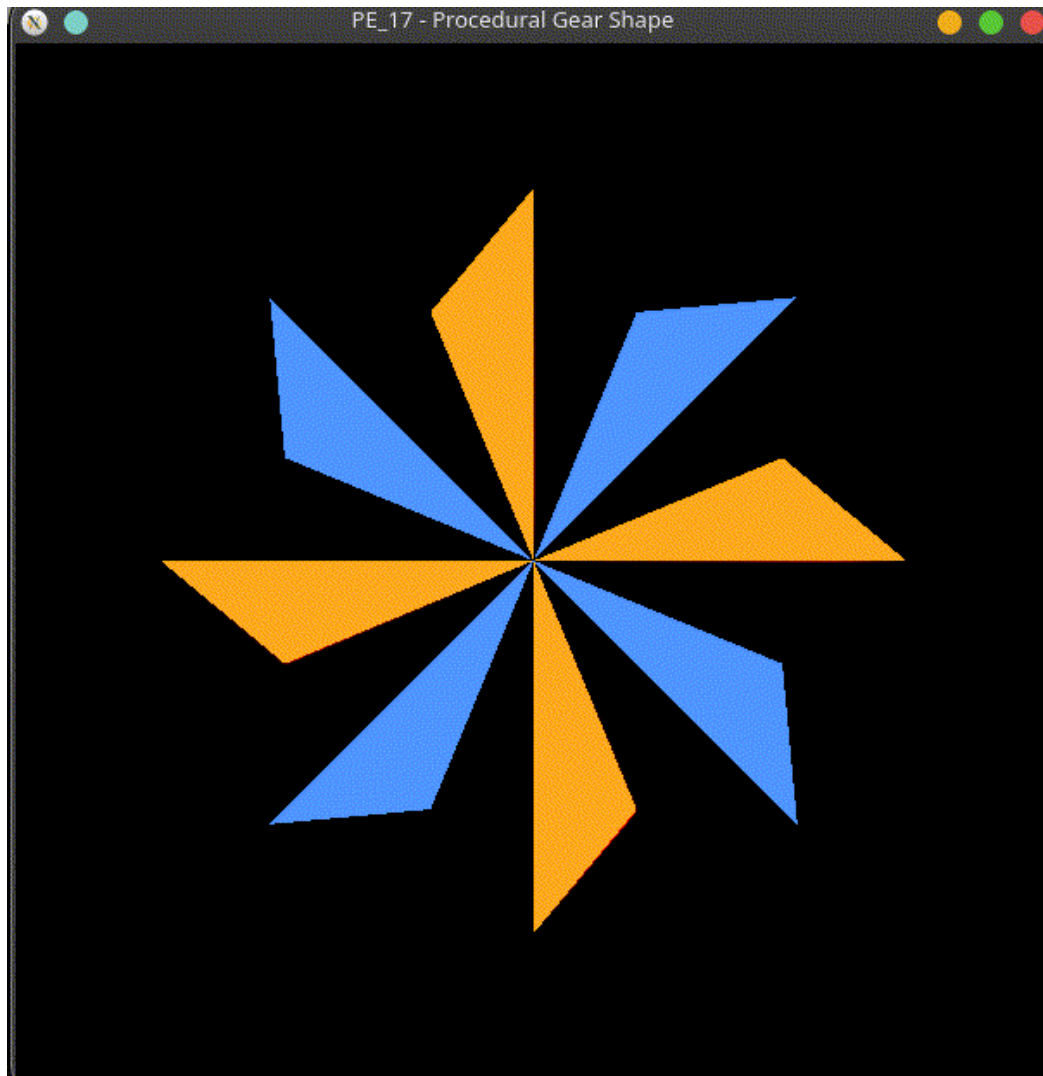
Objective: Practice generating a more complex indexed shape procedurally, with alternating colors across separate `glDrawElements` calls.

Task (Question): Build a procedurally generated ring of colored triangles (a simple 'gear' shape) using `glDrawElements`, where vertices are computed in a loop, and every other triangle uses a different color via TWO separate `glDrawElements` calls sharing ONE vertex pool.

Hints:

- Alternate between a larger radius and a smaller radius as you loop around the ring to create the 'teeth' look.

- Build two separate index arrays (one for 'even' teeth, one for 'odd' teeth) that both index into the SAME vertex array.



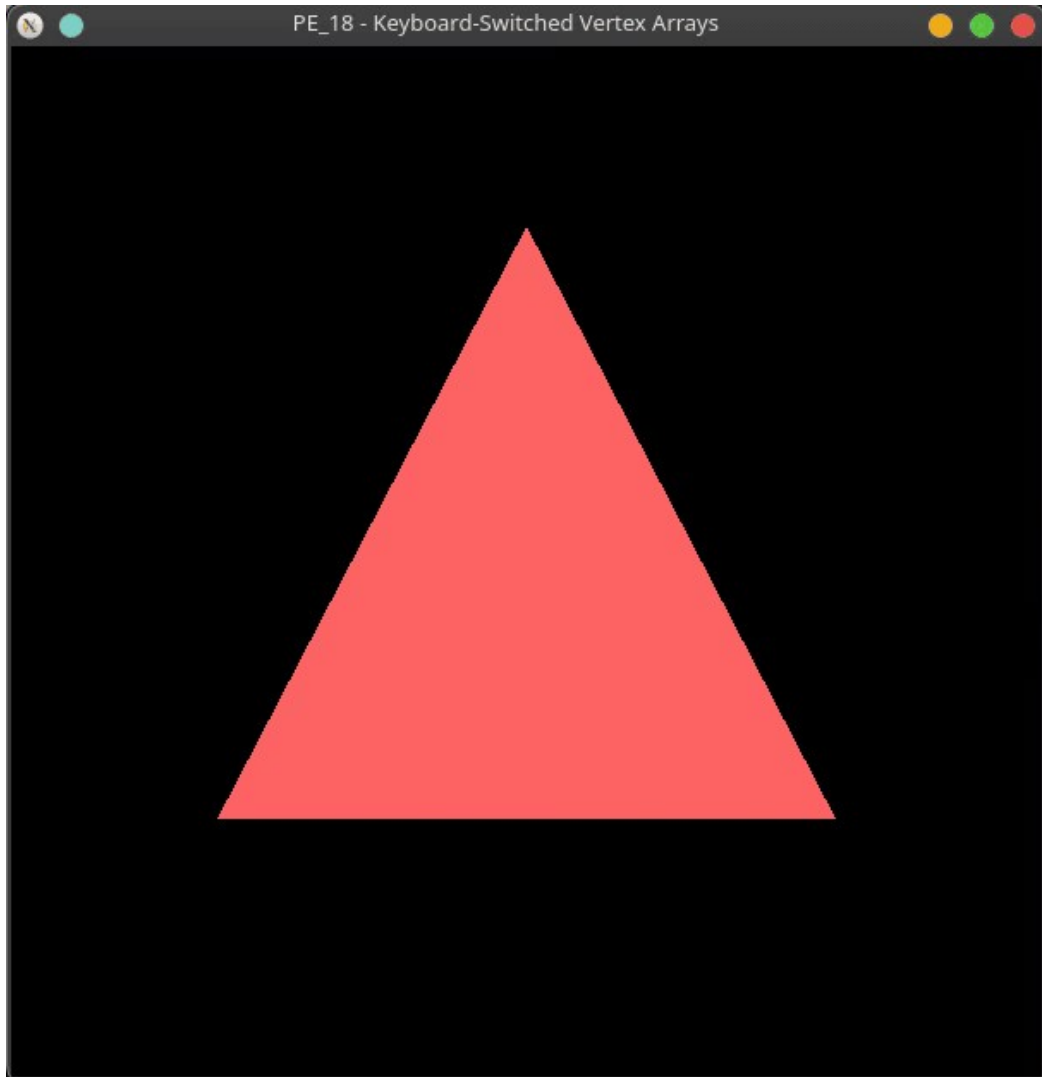
Exercise Q18 — Keyboard-Switched Vertex Arrays [Hard]

Objective: Practice combining vertex arrays with interactivity from Module 3.

Task (Question): Use `glutKeyboardFunc` so pressing '1', '2', or '3' switches which vertex array shape is displayed (a triangle, a quad, and a pentagon), WITHOUT rebuilding the arrays each time `display()` runs.

Hints:

- Declare all three shapes' vertex arrays as global data, built once, outside of `display()`.
- Store the current selection in a global int and branch on it inside `display()` to choose which array/mode to draw.



Exercise Q19 — Interleaved + Indexed Hexagon [Hard]

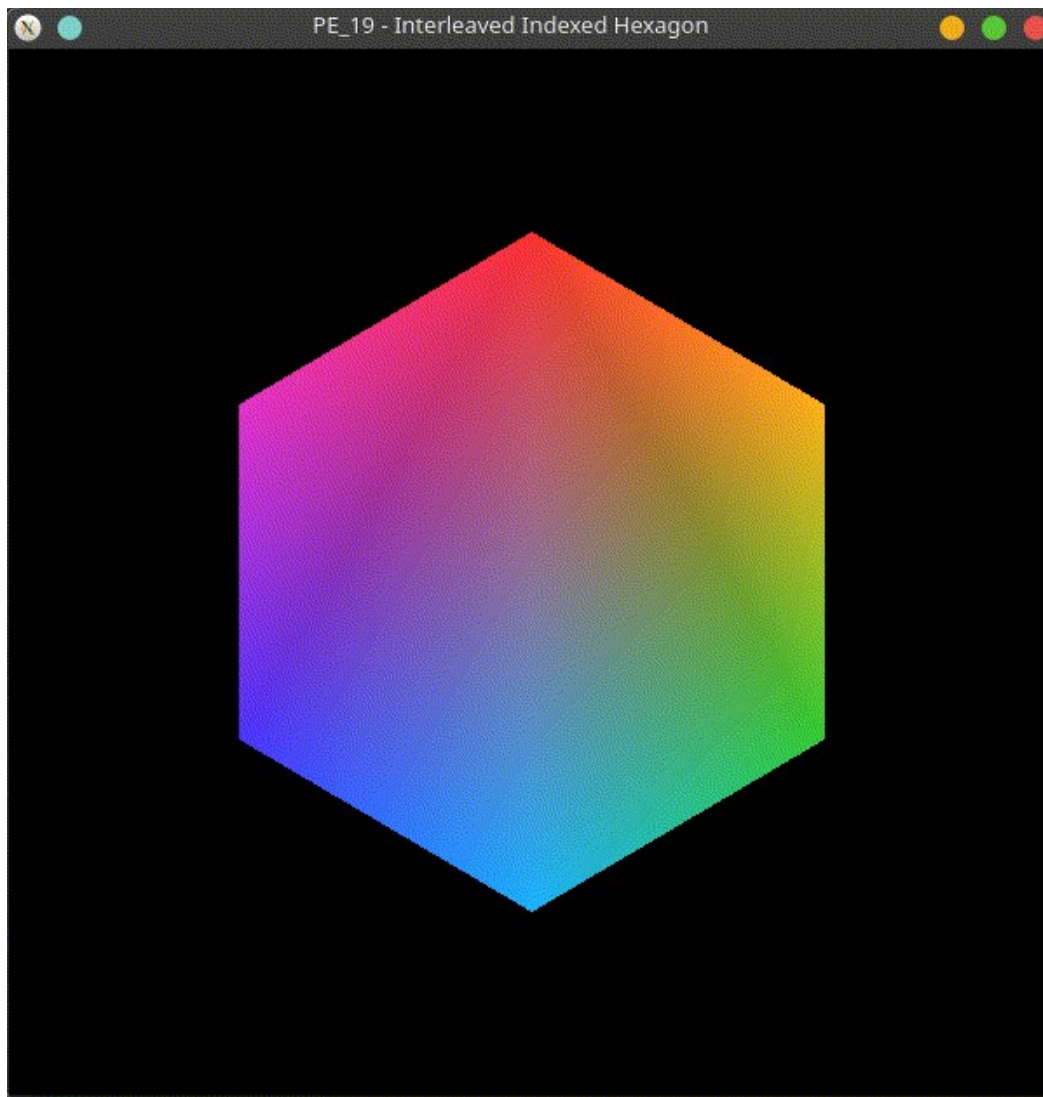
Objective: Practice combining EVERY vertex array technique from this module in a single shape.

Task (Question): Build an interleaved vertex+color array (position AND color in one array, like Example 16) for a hexagon rendered with `glDrawElements` AND a non-zero stride.

Hints:

- Six vertices, no shared/duplicate positions needed here — the index array can simply be `{0,1,2,3,4,5}` for `GL_POLYGON`.

- Double-check your stride math: with 6 floats per vertex, $\text{stride} = 6 * \text{sizeof}(\text{GLfloat})$, and the color pointer starts 3 floats after the vertex pointer.



Exercise Q20 — Capstone: Procedural Flower Scene [Hard]

Objective: Practice combining procedural generation, indexed rendering, color arrays, and multi-shape composition into one program.

Task (Question): Build a small procedurally generated 'flower' scene: a center disc (GL_TRIANGLE_FAN via `glDrawElements`), a ring of colored petals (GL_TRIANGLES via `glDrawArrays` with a vertex array + color array, generated in a loop, not hand-typed), and a ground quad.

Hints:

- Generate the petals' 3 vertices per petal (center point + two outer corners) inside a loop that walks around a circle by angle.
- Draw the petals FIRST, then the disc on top, so the disc doesn't get hidden behind petal geometry near

the center.

